



**Politechnika
Śląska**

PROJEKT INŻYNIERSKI

Projekt i implementacja aplikacji webowej do transkrypcji nagrań
fortepianowych na nuty w formacie PDF z wykorzystaniem
zaawansowanych modeli AI

Michał JAGODA

Nr albumu: 300405

Kierunek: Informatyka

Specjalność: Grafika Komputerowa i Oprogramowanie

PROWADZĄCY PRACĘ

dr hab. inż. Karolina Nurzyńska, Prof. Pol. Śl.

KATEDRA Algorytmiki i Oprogramowania

Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2025

Tytuł pracy

Projekt i implementacja aplikacji webowej do transkrypcji nagrań fortepianowych na nuty w formacie PDF z wykorzystaniem zaawansowanych modeli AI

Streszczenie

Niniejsza praca dotyczy opracowania i realizacji aplikacji webowej TuneScript, służącej do automatycznej transkrypcji nagrań fortepianowych do formatu partytur PDF. W projekcie wykorzystano zaawansowane modele sztucznej inteligencji, w szczególności sieci głębokie, przystosowane do detekcji nut i analizy sygnału audio. Aplikacja obejmuje moduł frontendowy, backendowy oraz specjalnie przygotowany moduł pracownika do asynchronicznego przetwarzania zadań. Głównym celem pracy była zarówno trafność rozpoznania nut, jak i czytelność generowanych partytur. Zrealizowany system, dzięki wykorzystaniu technik sztucznej inteligencji może być istotnym wsparciem dla muzyków, pedagogów oraz badaczy zainteresowanych automatycznym przetwarzaniem i analizą muzyki.

Słowa kluczowe

transkrypcja audio, sztuczna inteligencja, fortepian, partytura PDF, sieci neuronowe

Thesis title

Design and implementation of a web application for transcribing piano recordings into musical notes in PDF format using advanced AI models

Abstract

This thesis presents the design and implementation of a web application called TuneScript, which automatically transcribes piano recordings into musical scores in PDF format. The project leverages advanced artificial intelligence models, especially deep neural networks, adapted for note detection and audio signal analysis. The application is composed of a frontend, a backend, and a worker module for asynchronous task processing. The main objectives were to achieve both accurate note recognition and legible score generation. Thanks to the use of artificial intelligence techniques the resulting system can be a valuable tool for musicians, educators, and researchers interested in automatic music processing and analysis.

Key words

audio transcription, artificial intelligence, piano, PDF score, neural networks

Spis treści

1	Wstęp	1
2	Transkrypcja danych audio do zapisu nutowego	3
2.1	Sformułowanie problemu	3
2.2	Osadzenie tematu w kontekście aktualnego stanu wiedzy	4
2.3	Studia literaturowe	5
2.4	Analiza istniejących rozwiązań	7
2.5	Przegląd ważniejszych algorytmów	8
2.6	Proponowane rozwiązanie	9
3	Wymagania i narzędzia	11
3.1	Komponenty, moduły i biblioteki	11
3.2	Wymagania funkcjonalne	13
3.3	Wymagania нефункционалне	15
3.4	Wymagania sprzętowe i programowe	15
3.4.1	Wymagania dla użytkowników końcowych	16
3.4.2	Wymagania dla środowiska serwerowego	16
3.5	Przypadki użycia	16
4	Specyfikacja zewnętrzna	19
4.1	Wymagania sprzętowe i programowe	19
4.1.1	Wymagania programowe	19
4.2	Kategorie użytkowników	19
4.3	Sposób obsługi	20
4.4	Administracja systemem	20
4.5	Kwestie bezpieczeństwa	21
4.6	Przykład działania	21
4.6.1	Rozpoczęcie pracy z systemem	21
4.6.2	Panel główny i podstawowe funkcje	22
4.6.3	Proces transkrypcji	22
4.6.4	Zarządzanie transkrypcjami	24

4.6.5	Personalizacja i ustawienia	26
4.6.6	Wsparcie użytkownika	28
5	Specyfikacja wewnętrzna	29
5.1	Przedstawienie idei	29
5.2	Architektura systemu	29
5.3	Opis struktur danych i organizacji bazy danych	30
5.4	Skrypty deweloperskie	32
5.5	Szczegóły implementacji wybranych fragmentów	32
5.5.1	Implementacja modelu transkrypcji	32
5.5.2	Implementacja systemu kolejkowania transkrypcji	34
5.5.3	Implementacja systemu SSE do monitorowania postępu	34
5.6	Diagramy UML	34
6	Model transkrypcji	37
6.1	Architektura modelu	37
6.2	Proces uczenia	38
6.2.1	Przetwarzanie wstępne danych	38
6.2.2	Parametry treningu	39
6.2.3	Funkcja straty	39
6.3	Wnioskowanie	39
7	Weryfikacja i walidacja	41
7.1	Organizacja testów	41
7.1.1	Testy jednostkowe	41
7.1.2	Testy integracyjne	41
7.2	Przypadki testowe i wykryte błędy	42
7.2.1	Testy automatyczne	42
7.2.2	Testy manualne	45
7.3	Testowanie wydajności	46
7.4	Najważniejsze pokrycie testami	46
7.5	Porównanie wygenerowanych i oryginalnych partytur	47
7.5.1	Dokładność i ograniczenia transkrypcji	47
7.5.2	F. Chopin - Etiuda op. 10 nr 5	48
7.5.3	L. van Beethoven - Sonata op. 53 nr 21 "Waldstein", Allegro con brio	48
7.5.4	L. van Beethoven - Sonata op. 13 nr 8 "Patetyczna", Adagio cantabile	48
8	Podsumowanie i wnioski	55
8.1	Ocena osiągniętych celów	56
8.2	Ograniczenia i problemy napotkane w trakcie pracy	56

8.3	Możliwe kierunki dalszego rozwoju	57
8.4	Zakończenie	57
Bibliografia		61
Źródła		65
Spis rysunków		69
Spis tabel		71

Rozdział 1

Wstęp

W dobie cyfrowej transformacji muzyki, potrzeba efektywnych narzędzi do konwersji nagrań audio na zapis nutowy staje się coraz bardziej istotna. Niniejsza praca przedstawia projekt i implementację systemu TuneScript, który wykorzystuje zaawansowane modele sztucznej inteligencji do automatycznej transkrypcji nagrań fortepianowych. System ten stanowi odpowiedź na rosnące zapotrzebowanie muzyków, kompozytorów i pedagogów muzycznych na narzędzia umożliwiające szybką i dokładną konwersję nagrań audio do formatu partytur.

Problem automatycznej transkrypcji muzyki fortepianowej jest szczególnie złożony ze względu na polifoniczny charakter instrumentu oraz bogactwo ekspresji muzycznej. Dotychczasowe rozwiązania często nie spełniają oczekiwań użytkowników w zakresie dokładności transkrypcji lub wygody użytkowania. TuneScript adresuje te wyzwania poprzez zastosowanie najnowszych osiągnięć w dziedzinie uczenia maszynowego i przetwarzania sygnałów audio.

Głównym celem pracy jest stworzenie kompleksowego systemu, który nie tylko dokładnie rozpoznaje nuty i ich parametry czasowe, ale także generuje profesjonalnie wyglądające partytury w formacie Portable Document Format(PDF). Cele szczegółowe obejmują:

- Implementację modelu uczenia głębokiego do precyzyjnej detekcji nut i ich parametrów.
- Opracowanie systemu konwersji plików w formacie Musical Instrument Digital Interface (MIDI) do formatu PDF.
- Stworzenie intuicyjnego interfejsu webowego dla użytkowników.
- Zapewnienie mechanizmów zarządzania i udostępniania transkrypcji.

Zakres pracy obejmuje następujące obszary:

- Analiza istniejących rozwiązań i badań w dziedzinie automatycznej transkrypcji muzyki.

- Implementacja modelu uczenia maszynowego do transkrypcji.
- Projekt i implementacja systemu webowego.
- Integracja modelu uczenia maszynowego.
- Integracja mechanizmów generowania partytur.
- Testowanie i walidacja systemu.

Praca składa się z ośmiu rozdziałów. Po wstępie (rozdział 1), w rozdziale 2 przedstawiono szczegółową analizę problemu oraz przegląd literatury. Rozdział 3 opisuje wymagania funkcjonalne i нефункционалне oraz wykorzystane narzędzia. Rozdział 4 zawiera specyfikację zewnętrzną systemu, a rozdział 5 - specyfikację wewnętrzną wraz z opisem architektury i implementacji. W rozdziale 6 przedstawiono szczegóły modelu transkrypcji, a rozdział 7 opisuje proces weryfikacji i walidacji systemu. Pracę kończy podsumowanie i wnioski w rozdziale 8.

Rozdział 2

Transkrypcja danych audio do zapisu nutowego

W dzisiejszym świecie cyfrowa muzyka fortepianowa jest szeroko rozpowszechniona, a dostępność nagrań w formatach takich jak MPEG-1/MPEG-2 Audio Layer III (MP3) znacznie ułatwia ich odtwarzanie. Jednocześnie powstaje potrzeba narzędzi, które umożliwią analizę i edycję muzyki w sposób bardziej zaawansowany, szczególnie dla muzyków, nauczycieli i entuzjastów fortepianu. Niniejsza praca podejmuje próbę stworzenia kompleksowego rozwiązania, które przekształca nagrania audio w czytelne partytury, wspierając edukację, kompozycję i praktykę muzyczną.

2.1 Sformułowanie problemu

Współczesna muzyka fortepianowa często występuje w formie cyfrowych nagrań audio, najczęściej w formacie MP3. Format ten, choć wygodny do odtwarzania, nie pozwala na łatwą edycję czy analizę struktury muzycznej. Muzycy, kompozytorzy, pedagodzy muzyczni oraz entuzjaści fortepianu potrzebują narzędzia, które umożliwi im konwersję plików audio do formatu MIDI, a następnie do czytelnych partytur w formacie PDF.

Problem, którym zajmuje się niniejsza praca, polega na stworzeniu efektywnego, dokładnego i przyjaznego dla użytkownika systemu, który automatyzuje proces transkrypcji muzyki fortepianowej. System ten, nazwany TuneScript, ma za zadanie przekształcać nagrania fortepianowe z formatu MP3 do symbolicznej reprezentacji MIDI, a następnie generować profesjonalnie wyglądające partytury w formacie PDF.

Kluczowe wyzwania, które należy rozwiązać w ramach tego projektu, obejmują:

- Precyzyjne rozpoznawanie wysokości dźwięków w nagraniach fortepianowych, z uwzględnieniem złożoności polifonii fortepianowej.
- Dokładne określanie czasu rozpoczęcia i zakończenia każdej nuty, co jest kluczowe dla zachowania rytmu i tempa utworu.

- Odzworowanie dynamiki gry, która jest istotnym elementem ekspresji muzycznej na fortepianie.
- Efektywne przetwarzanie długich utworów muzycznych przy zachowaniu wysokiej jakości transkrypcji.
- Generowanie czytelnych i estetycznych partytur PDF, które będą użyteczne dla muzyków.

Rozwiązanie tego problemu ma potencjał znaczącego usprawnienia pracy muzyków, ułatwienia analizy muzycznej oraz wsparcia procesów edukacyjnych w dziedzinie muzyki fortepianowej.

2.2 Osadzenie tematu w kontekście aktualnego stanu wiedzy

Automatyczna transkrypcja muzyki (ATM, ang. *Automatic Music Transcription*) jest jednym z najbardziej złożonych problemów w dziedzinie przetwarzania sygnałów muzycznych (MIR, ang. *Music Information Retrieval*). W kontekście muzyki fortepianowej, ATM zyskuje szczególne znaczenie ze względu na popularność tego instrumentu oraz jego rolę w edukacji muzycznej i kompozycji.

Obecny stan wiedzy w dziedzinie ATM dla fortepianu charakteryzuje się następującymi aspektami:

1. **Wykorzystanie głębokiego uczenia:** Współczesne podejścia do ATM fortepianowego opierają się głównie na zastosowaniu głębokich sieci neuronowych. Modele takie jak konwolucyjne sieci neuronowe (CNN) i rekurencyjne sieci neuronowe (RNN) wykazały znaczącą skuteczność w rozpoznawaniu struktur muzycznych w spektrogramach audio [10].
2. **Modele hybrydowe:** Najnowsze badania skupiają się na modelach hybrydowych, łączących różne architektury sieci neuronowych. Na przykład, model “Onsets and Frames” [10] wykorzystuje osobne sieci do detekcji początków dźwięków i ich trwania, co znacząco poprawiło dokładność transkrypcji.
3. **Przetwarzanie w czasie rzeczywistym:** Rozwój technik przetwarzania sygnałów i optymalizacji algorytmów pozwala na coraz szybszą transkrypcję, zbliżającą się do przetwarzania w czasie rzeczywistym [14].
4. **Modelowanie akustyki fortepianu:** Zaawansowane modele uwzględniają specyfikę akustyki fortepianu, w tym zjawiska takie jak rezonans strun i wpływ pedałów na brzmienie [4].

5. **Transkrypcja ekspresji muzycznej:** Nowsze badania skupiają się nie tylko na rozpoznawaniu nut, ale także na odwzorowaniu dynamiki, artykulacji i innych aspektów ekspresji muzycznej [13].

Mimo znaczących postępów w dziedzinie ATM dla fortepianu, wciąż istnieją pewne ograniczenia i wyzwania:

- **Dokładność w muzyce polifonicznej:** Transkrypcja złożonych struktur polifonicznych, charakterystycznych dla zaawansowanej literatury fortepianowej, wciąż stanowi wyzwanie dla obecnych systemów [2].
- **Rozpoznawanie bardzo cichych dźwięków:** Detekcja nut granych bardzo cicho, szczególnie w kontekście głośniejszego akompaniamentu, pozostaje problematyczna [29].
- **Odróżnianie bliskich harmonicznie dźwięków:** W gęstych strukturach harmonicznych, rozróżnienie blisko brzmiących dźwięków może być trudne dla systemów ATM [2].
- **Interpretacja rytmiczna:** Precyzyjne odwzorowanie złożonych struktur rytmicznych, szczególnie w muzyce z rubato lub improwizowanej, stanowi wyzwanie dla obecnych systemów [4].
- **Wykorzystanie pedałów:** Uwzględnienie użytkowania pedałów fortepianowych, które wpływają na rezonans strun i brzmienie, wymaga zaawansowanych technik analizy akustycznej [4].

Projekt TuneScript wpisuje się w ten kontekst, stawiając sobie za cel adresowanie wymienionych wyzwań poprzez integrację zaawansowanego modelu sztucznej inteligencji (AI, ang. *Artificial Intelligence*) do transkrypcji z MP3 do MIDI z narzędziami do generowania partytur PDF. Takie kompleksowe podejście ma potencjał przewyższenia niektórych ograniczeń obecnych rozwiązań i dostarczenia użytkownikom narzędzia o wysokiej praktycznej wartości.

2.3 Studia literaturowe

W celu głębszego zrozumienia problematyki automatycznej transkrypcji muzyki fortepianowej oraz określenia aktualnego stanu wiedzy w tej dziedzinie, przeprowadzono szczegółowy przegląd literatury. Poniżej przedstawiono kluczowe prace i ich wkład w rozwój tej dziedziny:

1. **Kong et al. (2020)** [15] w pracy “High-resolution Piano Transcription with Pedals by Regressing Onsets and Offsets Times” prezentują nowatorskie podejście do transkrypcji fortepianowej, które stanowi podstawę modelu wykorzystywanego w projekcie TuneScript. Autorzy proponują metodę wysokiej rozdzielczości czasowej do transkrypcji fortepianowej, która uwzględnia nie tylko detekcję nut, ale także użycie pedałów. Kluczową innowacją jest zastosowanie regresji do precyzyjnego określania czasów początku (onset) i końca (offset) każdej nuty, co pozwala na dokładniejsze odwzorowanie rytmu i artykulacji. Model ten wykazuje znaczącą poprawę w stosunku do wcześniejszych rozwiązań, szczególnie w zakresie precyzji czasowej i detekcji użycia pedałów.
2. **Benetos et al. (2013)** [2] w swojej pracy przeglądowej “Automatic music transcription: challenges and future directions” przedstawiają kompleksowe omówienie technik ATM. Autorzy omawiają główne wyzwania w dziedzinie, takie jak polifonia, różnorodność instrumentów i style wykonania. W kontekście muzyki fortepianowej, praca ta podkreśla złożoność problemu transkrypcji ze względu na szeroką gamę dynamiczną i bogactwo harmoniczne fortepianu.
3. **Hawthorne et al. (2018)** [10] w artykule “Onsets and frames: Dual-objective piano transcription” prezentują przełomowy model do transkrypcji fortepianowej. Ich podejście, nazwane “Onsets and Frames”, wykorzystuje osobne sieci neuronowe do detekcji początków dźwięków i ich trwania. Model ten znacząco poprawił dokładność transkrypcji fortepianowej, osiągając wyniki bliskie ludzkiej dokładności w niektórych aspektach. Jest to jedno z najbardziej zaawansowanych rozwiązań w dziedzinie ATM dla fortepianu, które stanowiło inspirację dla późniejszych prac, w tym modelu Kong et al.
4. **Kelz et al. (2016)** [14] w pracy “On the Potential of Simple Framewise Approaches to Piano Transcription” badają możliwości prostszych podejść do transkrypcji fortepianowej, opierających się na analizie pojedynczych ramek czasowych. Autorzy wykazują, że nawet stosunkowo proste modele mogą osiągać konkurencyjne wyniki, co ma istotne implikacje dla projektowania systemów działających w czasie rzeczywistym.
5. **Cheng et al. (2016)** [4] w artykule “Attack/decay analysis of piano sounds using an exponential model” skupiają się na modelowaniu akustycznych właściwości dźwięków fortepianowych. Ich praca jest istotna dla zrozumienia, jak precyzyjnie analizować początkową fazę dźwięku (atak) oraz jego wygaszanie (decay), co jest kluczowe dla dokładnej transkrypcji dynamiki i artykulacji.
6. **Jeong et al. (2017)** [13] w pracy “Deep Learning for Music” prezentują szerokie zastosowanie technik głębokiego uczenia w różnych zadaniach związanych z przetwa-

rzaniem muzyki, w tym w transkrypcji fortepianowej. Autorzy omawiają, jak różne architektury sieci neuronowych mogą być wykorzystane do modelowania różnych aspektów muzyki, od wysokości dźwięku po ekspresję.

7. **Sigtia et al. (2016) [29]** w pracy “An end-to-end neural network for polyphonic piano music transcription” prezentują kompleksowe podejście do transkrypcji muzyki fortepianowej wykorzystujące głębokie sieci neuronowe. Ich model jest w stanie bezpośrednio przetwarzać surowy sygnał audio na notację muzyczną, co stanowi znaczący krok w kierunku w pełni automatycznej transkrypcji.

Powyższe prace stanowią fundament teoretyczny dla projektu TuneScript. Pokazują one ewolucję technik ATM dla fortepianu od tradycyjnych metod przetwarzania sygnałów do zaawansowanych modeli uczenia głębokiego. Jednocześnie, literatura ta wskazuje na wciąż istniejące wyzwania, szczególnie w zakresie transkrypcji muzyki polifonicznej, dokładnego odwzorowania dynamiki i ekspresji oraz generowania wysokiej jakości partytur.

Projekt TuneScript czerpie z tych badań, integrując najnowsze osiągnięcia w dziedzinie ATM z praktycznym aspektem generowania użytecznych partytur. Szczególnie istotna jest praca Kong et al. [15], która stanowi podstawę modelu transkrypcji wykorzystywanego w systemie. Model ten, dzięki swojej wysokiej rozdzielczości czasowej i uwzględnieniu pedałów, pozwala na bardziej precyzyjną transkrypcję niż wcześniejsze rozwiązania.

Warto zauważyć, że mimo znaczącego postępu w dziedzinie ATM dla fortepianu, wciąż istnieje przestrzeń do dalszych ulepszeń, szczególnie w zakresie dokładności transkrypcji bardzo złożonych utworów polifonicznych oraz wiernego odwzorowania subtelności ekspresji muzycznej. TuneScript stara się adresować te wyzwania, oferując kompleksowe rozwiązanie łączące zaawansowaną transkrypcję z praktycznym zastosowaniem w postaci generowania partytur.

2.4 Analiza istniejących rozwiązań

W ramach analizy rynku przeprowadzono szczegółowe badanie istniejących rozwiązań do automatycznej transkrypcji muzyki fortepianowej. Proces poszukiwań obejmował systematyczny przegląd platform dystrybucji oprogramowania, w tym Google Play oraz App Store, a także analizę dostępnych rozwiązań na komputery osobiste (PC, ang. Personal Computer). Dodatkowo przeanalizowano repozytoria kodu źródłowego takie jak GitHub i GitLab, przestudiowano literaturę branżową oraz fora muzyczne, a także przeprowadzono szczegółowe wyszukiwanie w internecie przy użyciu fraz kluczowych w różnych językach.

W wyniku analizy zidentyfikowano kilka znaczących rozwiązań komercyjnych. Pierwszym z nich jest AnthemScore (<https://www.lunaverus.com/>), aplikacja desktopowa służąca do transkrypcji audio na nuty. Mimo że oferuje podstawową funkcjonalność transkrypcji, produkt ten ma szereg istotnych ograniczeń. Przede wszystkim brakuje mu wersji webowej, co znacząco

ogranicza dostępność. Ponadto aplikacja wykazuje ograniczoną dokładność w przypadku muzyki polifonicznej, stawia wysokie wymagania sprzętowe oraz nie oferuje automatycznej detekcji pedalizacji.

Kolejnym znaczącym rozwiązaniem jest ScoreCloud (<https://scorecloud.com/>), system umożliwiający transkrypcję zarówno audio jak i MIDI. System ten jednak zmaga się z istotnymi problemami w rozpoznawaniu złożonych struktur harmoniczných oraz wykazuje ograniczoną dokładność rytmiczną. Co więcej, brak specjalizacji pod fortepian oraz wysoki koszt subskrypcji stanowią znaczące bariery dla potencjalnych użytkowników.

Na rynku obecny jest również Melodyne (<https://www.celemony.com/en/melodyne/>), profesjonalne narzędzie do edycji audio. Choć jest to zaawansowane oprogramowanie, jego główny nacisk położony jest na edycję dźwięku, nie na transkrypcję. Skomplikowany interfejs i bardzo wysoka cena czynią go mało dostępnym dla przeciętnego użytkownika, a brak automatycznego generowania partytur PDF znacząco ogranicza jego przydatność w kontekście transkrypcji fortepianowej.

Wśród rozwiązań darmowych wyróżnia się Audacity (<https://www.audacityteam.org/>) z wtyczką AudioToScore. To rozwiązanie open-source, mimo że dostępne bez opłat, charakteryzuje się niską dokładnością transkrypcji, brakiem odpowiedniego wsparcia dla złożonej polifonii, problematycznym interfejsem użytkownika oraz ograniczonymi możliwościami eksportu.

Przeprowadzona analiza jasno wskazuje na istotne luki w obecnych rozwiązaniach rynkowych. Główne problemy to brak specjalizacji pod muzykę fortepianową, niedostateczna dokładność w przypadku muzyki polifonicznej, brak integracji z nowoczesnymi modelami AI, ograniczona dostępność wynikająca z koncentracji na aplikacjach desktopowych zamiast webowych, oraz wysokie koszty licencji.

W tym kontekście TuneScript wprowadza istotną wartość na rynek poprzez kilka kluczowych innowacji. System wykorzystuje najnowsze modele AI specjalnie dostosowane do specyfiki fortepianu, oferuje wygodny dostęp przez przeglądarkę internetową, oraz wprowadza automatyczną detekcję pedalizacji. Szczególny nacisk położono na dokładność transkrypcji muzyki polifonicznej oraz generowanie profesjonalnie wyglądających partytur PDF, co bezpośrednio odpowiada na główne ograniczenia zidentyfikowane w istniejących rozwiązaniach.

2.5 Przegląd ważniejszych algorytmów

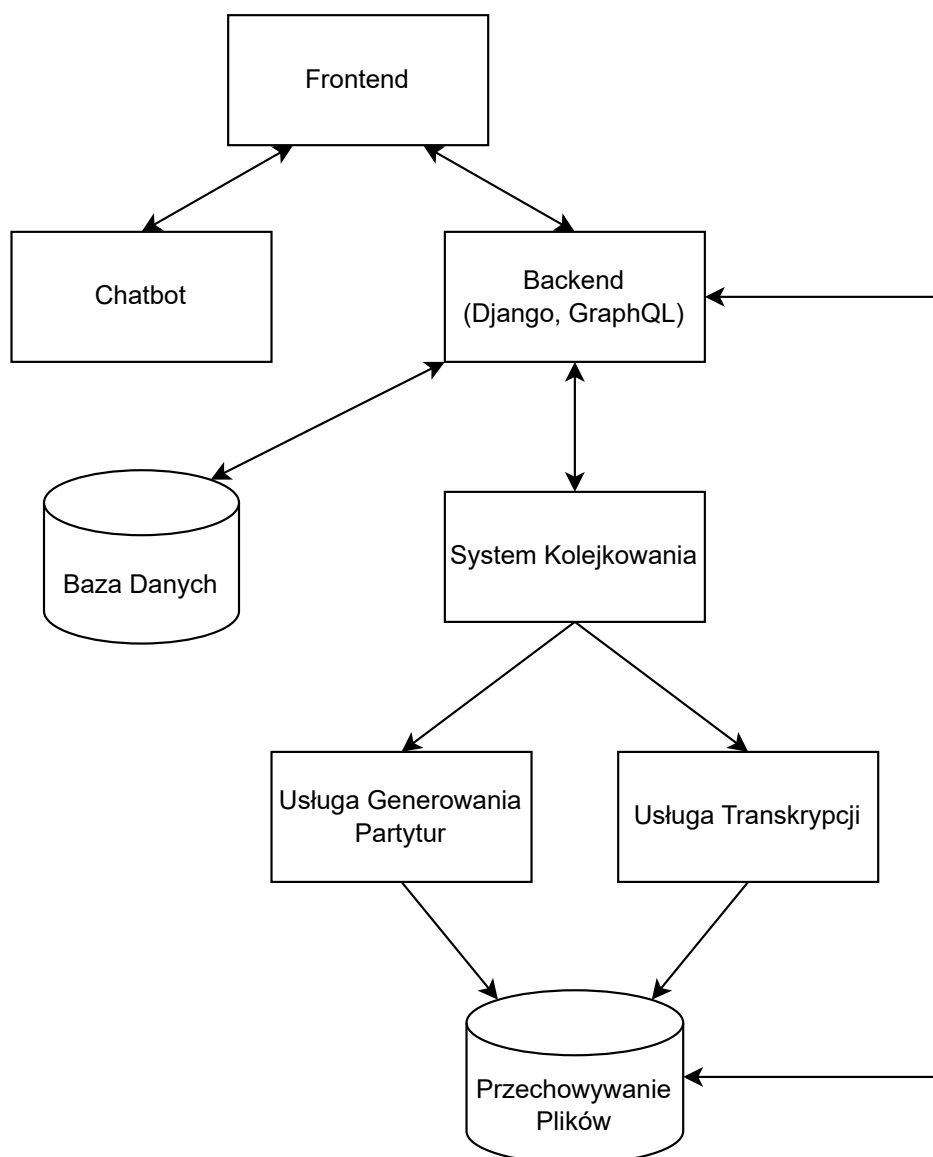
Kluczowym algorytmem w systemie TuneScript jest model transkrypcji muzycznej oparty na pracy Kong et al. [15]. Główne etapy tego algorytmu to:

1. Przetwarzanie wstępne sygnału audio (np. normalizacja, konwersja do spektrogramu).
2. Analiza sygnału przez sieć konwolucyjną w celu wykrycia początków (onset) i końców (offset) nut.
3. Regresja czasów początku i końca każdej nuty.
4. Detekcja użycia pedałów fortepianu.

5. Generowanie transkrypcji MIDI na podstawie wykrytych nut i pedałów.

2.6 Proponowane rozwiązanie

Na podstawie przeprowadzonej analizy literatury oraz istniejących rozwiązań, zaproponowano system TuneScript, którego architekturę przedstawiono na rysunku 2.1. System został zaprojektowany w architekturze mikroserwisowej, co zapewnia elastyczność rozwoju oraz niezależne skalowanie poszczególnych komponentów.



Rysunek 2.1: Architektura systemu TuneScript

Zaproponowana architektura adresuje zidentyfikowane problemy poprzez szereg rozwiązań

technicznych. System został zaimplementowany jako aplikacja webowa, co zapewnia szerszą dostępność niż rozwiązania desktopowe. Do transkrypcji wykorzystano zaawansowane modele sztucznej inteligencji, a przetwarzanie długich utworów odbywa się w sposób asynchroniczny. Ponadto, system oferuje automatyczną detekcję pedalizacji oraz generowanie profesjonalnych partytur w formacie PDF.

Rozdział 3

Wymagania i narzędzia

Rozdział ten przedstawia przegląd wymagań i narzędzi niezbędnych do realizacji systemu TuneScript. Omówione zostaną zarówno wymagania funkcjonalne i нефункционалне, jak również wymagania sprzętowe i programowe oraz wykorzystane technologie i narzędzia. Szczególną uwagę poświęcono doborowi odpowiednich komponentów i bibliotek, które umożliwiają efektywną implementację złożonych funkcjonalności systemu.

3.1 Komponenty, moduły i biblioteki

Realizacja systemu TuneScript wymaga wykorzystania szeregu nowoczesnych technologii i narzędzi programistycznych. Wybór konkretnych rozwiązań został podyktowany specyficznymi wymaganiami projektu, takimi jak wydajne przetwarzanie dźwięku, skalowalność systemu oraz zapewnienie intuicyjnego interfejsu użytkownika. Poniżej przedstawiono główne komponenty i biblioteki wraz z uzasadnieniem ich wyboru:

- **Frontend:**
 - Next.js [32]: Framework React wybrany ze względu na:
 - * Wydajne renderowanie po stronie serwera (SSR, ang. *Server-Side Rendering*).
 - * Wbudowaną optymalizację SEO (ang. *Search Engine Optimization*, optymalizacja dla wyszukiwarek internetowych).
 - * Automatyczny routing i dzielenie kodu (ang. *code splitting*).
 - * Wsparcie dla API (ang. *Application Programming Interface*, interfejs programowania aplikacji) Routes ułatwiające integrację z backendem.
 - TypeScript [18]: Rozszerzenie JavaScript zastosowane dla:
 - * Wczesnego wykrywania błędów dzięki statycznemu typowaniu.
 - * Lepszej dokumentacji kodu poprzez interfejsy i typy.
 - * Udoskonalonego wsparcia IDE (ang. *Integrated Development Environment*, zintegrowane środowisko programistyczne) i autouzupełniania.

- Apollo Client [1]: Klient GraphQL (ang. *Graph Query Language*) wybrany ze względu na:
 - * Zaawansowane zarządzanie stanem aplikacji.
 - * Inteligentne cachowanie zapytań.
 - * Wsparcie dla TypeScript.
- **Backend:**
 - Python [26]: Główny język programowania wybrany ze względu na:
 - * Rozbudowany ekosystem bibliotek do uczenia maszynowego (ML, ang. *Machine Learning*).
 - * Wydajne przetwarzanie danych numerycznych.
 - * Czytelną składnię ułatwiającą rozwój projektu.
 - Django [6]: Framework webowy zastosowany dla:
 - * Solidnego systemu autentykacji i autoryzacji.
 - * ORM (ang. *Object-Relational Mapping*, mapowanie obiektowo-relacyjne) ułatwiającego pracę z bazą danych.
 - * Wbudowanych mechanizmów bezpieczeństwa.
 - Flask [19]: Lekki framework webowy wykorzystany dla:
 - * Implementacji serwisu chatbota.
 - * Prostej integracji z modelami uczenia maszynowego.
 - * Elastyczności w tworzeniu mikrouslug.
 - Celery [3] z Redis [28]: System kolejkowania wybrany dla:
 - * Asynchronicznego przetwarzania długotrwałych zadań.
 - * Efektywnej dystrybucji obciążenia.
 - * Niezawodnej komunikacji między komponentami.
- **Komponenty ML:**
 - PyTorch [27]: Framework ML (ang. *Machine Learning*, uczenie maszynowe) wybrany ze względu na:
 - * Elastyczność w definiowaniu modeli neuronowych.
 - * Wydajne przetwarzanie na GPU (ang. *Graphics Processing Unit*, procesor graficzny).
 - * Rozbudowane narzędzia do treningu modeli.
 - Transformers [12]: Biblioteka NLP (ang. *Natural Language Processing*, przetwarzanie języka naturalnego) zastosowana dla:
 - * Dostępu do najnowszych modeli językowych.
 - * Gotowych implementacji architektury Transformer.

- * Łatwej integracji z PyTorch.

- **Infrastruktura:**

- PostgreSQL [20]: System bazodanowy wybrany ze względu na:
 - * Wsparcie dla złożonych zapytań i indeksów.
 - * Niezawodność i skalowalność.
 - * Dobrą integrację z Django.
- Docker [7] z Compose: Narzędzia konteneryzacji zastosowane dla:
 - * Spójnego środowiska deweloperskiego.
 - * Łatwego zarządzania mikrousługami.
 - * Uproszczonego wdrażania aplikacji.

Powyższa struktura technologiczna została zaprojektowana z myślą o zapewnieniu wydajności, skalowalności i łatwości utrzymania systemu. Wybrane narzędzia reprezentują sprawdzone rozwiązania w swoich dziedzinach, jednocześnie oferując nowoczesne podejście do tworzenia aplikacji webowych wspierających uczenie maszynowe.

Mimo że projekt był realizowany indywidualnie, zdecydowano się na wdrożenie profesjonalnych narzędzi deweloperskich. Jest to zgodne z dobrymi praktykami inżynierii oprogramowania i ułatwia potencjalny przyszły rozwój systemu. W projekcie wykorzystano następujące narzędzia:

- **Zarządzanie zależnościami i jakością kodu:**

- Poetry [24]: Nowoczesne zarządzanie zależnościami Python.
- ESLint [8] i Prettier [21]: Spójne formatowanie kodu JavaScript/TypeScript.
- Black [25] i isort[31]: Automatyczne formatowanie kodu Python.
- Flake8 [22]: Analiza statyczna kodu Python.

- **Testowanie:**

- Pytest [23]: Kompleksowe testowanie komponentów Pythona.
- Jest [17]: Testy jednostkowe i integracyjne frontendu.
- Factory Boy [9]: Generowanie danych testowych.
- Cypress [5]: Testy end-to-end interfejsu użytkownika.

3.2 Wymagania funkcjonalne

Wymagania funkcjonalne systemu TuneScript definiują konkretne funkcje i możliwości, które system musi zapewniać użytkownikom. Zostały one opracowane na podstawie analizy potrzeb docelowych użytkowników oraz specyfiki procesu transkrypcji muzycznej. System musi zapewniać kompletny zestaw funkcjonalności umożliwiających zarówno pracę z transkrypcjami, jak i interakcję między użytkownikami. Poniżej przedstawiono szczegółowy wykaz wymagań funkcjonalnych:

1. Uwierzytelnianie i autoryzacja użytkowników:

- System musi zapewniać funkcjonalności rejestracji, logowania i wylogowywania użytkowników.
- Użytkownicy muszą mieć możliwość resetowania haseł za pomocą poczty elektronicznej.

2. Przesyłanie plików:

- System musi umożliwiać użytkownikom przysyłanie plików audio w formatach MP3 i WAV (ang. *Waveform Audio File Format*, format pliku audio typu wave).
- System musi weryfikować format przesyłanych plików.

3. Przetwarzanie transkrypcji:

- System musi przetwarzać przesłane pliki audio w celu generowania transkrypcji fortepianowych.
- Transkrypcje powinny być zapisywane w formacie MIDI oraz generowane jako partytury PDF.

4. Zarządzanie transkrypcjami:

- Użytkownicy muszą mieć możliwość przeglądania, edytowania i usuwania swoich transkrypcji.
- Użytkownicy muszą mieć możliwość ustawiania widoczności swoich transkrypcji jako publicznych lub prywatnych.

5. Funkcjonalność wyszukiwania:

- System musi zapewniać możliwości wyszukiwania transkrypcji na podstawie tytułu, kompozytora i widoczności.

6. System oceniania:

- Użytkownicy muszą mieć możliwość oceniania transkrypcji.
- System musi obliczać i wyświetlać średnią ocenę dla każdej transkrypcji.

7. Zarządzanie ulubionymi:

- Użytkownicy muszą mieć możliwość dodawania transkrypcji do ulubionych.
- Użytkownicy muszą mieć możliwość przeglądania i zarządzania swoimi ulubionymi transkrypcjami.

3.3 Wymagania niefunkcjonalne

W celu zapewnienia wysokiej jakości działania systemu, oprócz wymagań funkcjonalnych, zdefiniowano szereg wymagań niefunkcjonalnych. Określają one cechy jakościowe systemu, takie jak wydajność, niezawodność czy bezpieczeństwo. Poniżej przedstawiono kluczowe wymagania niefunkcjonalne:

1. **Wydajność:** Wydajność systemu jest kluczowa dla zapewnienia dobrego doświadczenia użytkownika. System powinien spełniać następujące kryteria:

- System musi przetwarzać transkrypcje w rozsądnych ramach czasowych (np. mniej niż 5 minut dla 10-minutowego pliku audio).
- System powinien obsługiwać jednocześnie co najmniej 100 aktywnych użytkowników.

2. **Użyteczność i doświadczenie użytkownika:**

- Interfejs użytkownika musi być intuicyjny i łatwy w nawigacji.
- System musi dostarczać użytkownikom jasne instrukcje i informacje zwrotne.

3. **Niezawodność i dostępność:**

- System musi być niezawodny, z dostępnością na poziomie 99,9% (nie licząc planowanych przerw technicznych).
- System musi zapewniać mechanizmy automatycznego odzyskiwania po awariach.

4. **Bezpieczeństwo i prywatność:**

- System musi chronić dane użytkowników za pomocą szyfrowania.
- System musi wykorzystywać bezpieczne mechanizmy uwierzytelniania.

5. **Kompatybilność i dostępność:**

- System musi być kompatybilny z przeglądarkami Chrome (wersja 90+), Firefox (wersja 88+), Safari (wersja 14+) i Edge (wersja 90+).
- System musi działać na systemach Windows 10+, macOS 10.15+, Android 9.0+ oraz iOS 14.0+.
- Interfejs użytkownika powinien być responsywny i działać na urządzeniach mobilnych przy minimalnej rozdzielczości 320x568.

3.4 Wymagania sprzętowe i programowe

Prawidłowe funkcjonowanie systemu TuneScript wymaga spełnienia określonych wymagań technicznych, zarówno po stronie klienta, jak i serwera. Wymagania te zostały zoptymalizowane tak, aby zapewnić szeroką dostępność systemu przy jednoczesnym zachowaniu wysokiej jakości działania.

3.4.1 Wymagania dla użytkowników końcowych

System został zaprojektowany z myślą o minimalnych wymaganiach sprzętowych dla użytkowników końcowych:

- Urządzenie z dostępem do internetu (komputer, tablet lub smartfon).
- Nowoczesna przeglądarka internetowa (Chrome, Firefox, Safari lub Edge w najnowszej wersji).
- Minimum 4 GB RAM dla optymalnego działania aplikacji.
- Stabilne połączenie internetowe o prędkości co najmniej 5 Mbps.

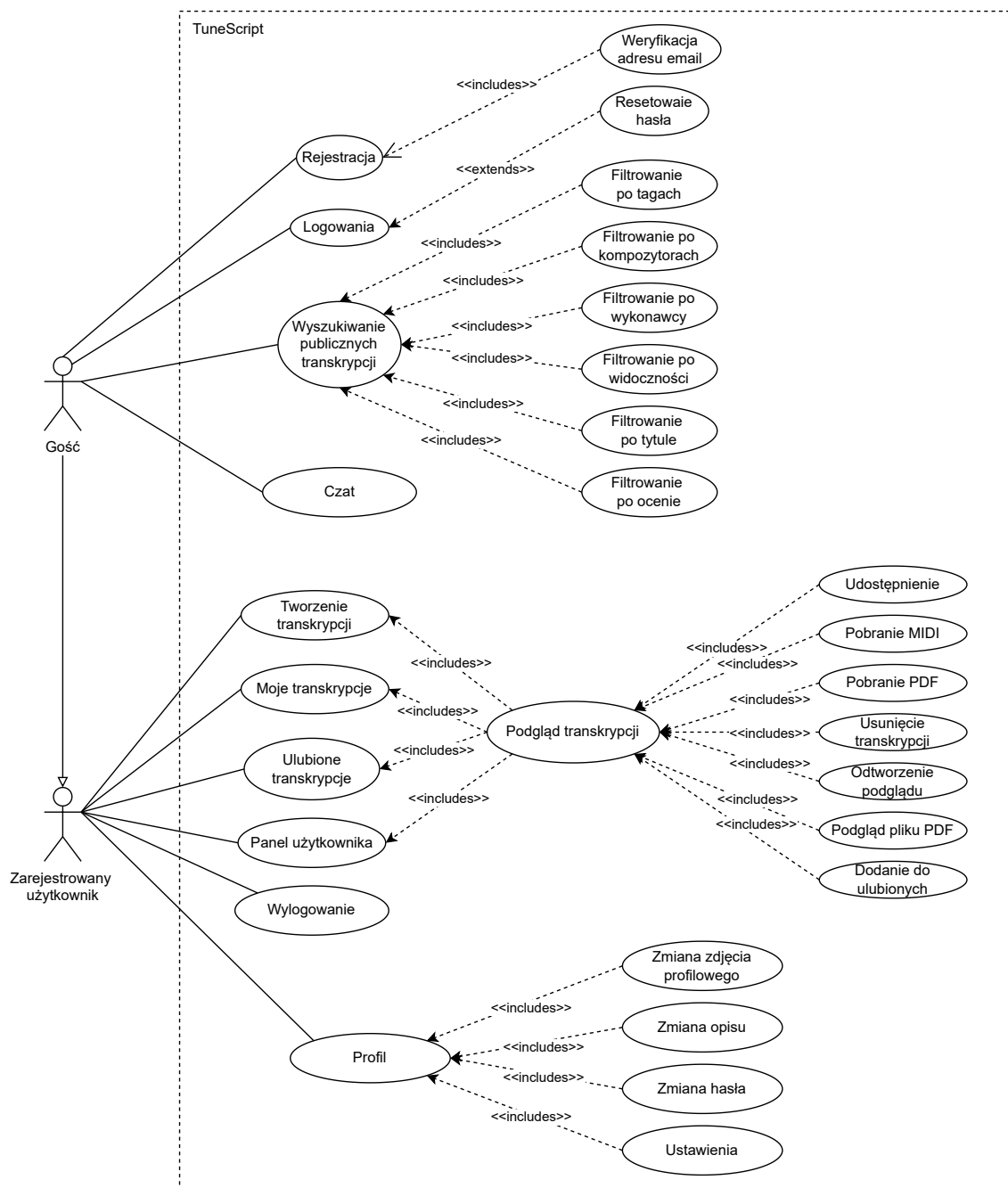
3.4.2 Wymagania dla środowiska serwerowego

Dla zapewnienia płynnej pracy systemu, serwer musi spełniać następujące wymagania:

- Procesor: minimum 4 rdzenie, zalecane 8 rdzeni.
- RAM: minimum 16 GB, zalecane 32 GB.
- Dysk: minimum 100 GB przestrzeni dyskowej SSD.
- System operacyjny: Linux (zalecany Ubuntu 20.04 LTS lub nowszy).
- Stabilne połączenie internetowe o wysokiej przepustowości.

3.5 Przypadki użycia

Zdefiniowane przypadki użycia przedstawiają typowe interakcje użytkowników z systemem TuneScript. Dokumentacja przypadków użycia pozwala na lepsze zrozumienie sposobu, w jaki różni użytkownicy będą korzystać z systemu oraz jakie funkcjonalności muszą być dostępne dla poszczególnych grup użytkowników.



Rysunek 3.1: Diagram przypadków użycia systemu TuneScript

Jak przedstawiono na rysunku 3.1, kluczowe przypadki użycia obejmują:

- **Rejestracja i uwierzytelnianie:** Obejmuje proces tworzenia konta, logowania i zarządzania dostępem do systemu.
- **Zarządzanie transkrypcjami:** Dotyczy przesyłania plików audio, generowania transkrypcji oraz zarządzania wygenerowanymi partyturami.

- **Interakcje społecznościowe:** Obejmuje ocenianie transkrypcji, dodawanie ich do ulubionych oraz udostępnianie innym użytkownikom.
- **Wyszukiwanie i przeglądanie:** Dotyczy możliwości przeszukiwania bazy transkrypcji i przeglądania dostępnych materiałów.
- **Zarządzanie kontem:** Obejmuje funkcje związane z personalizacją ustawień konta i preferencji użytkownika.

Rozdział 4

Specyfikacja zewnętrzna

Rozdział ten przedstawia specyfikację zewnętrzną systemu TuneScript, opisując interfejsy użytkownika, scenariusze korzystania z systemu oraz wymagania dotyczące integracji z innymi systemami. Specyfikacja zewnętrzna ma na celu zaprezentowanie funkcjonalności systemu z perspektywy użytkownika, uwzględniając różne przypadki użycia i interakcje z systemem.

4.1 Wymagania sprzętowe i programowe

System TuneScript został zaprojektowany jako aplikacja webowa działająca w architekturze klient-serwer. Poniżej przedstawiono minimalne i zalecane wymagania sprzętowe oraz programowe niezbędne do prawidłowego funkcjonowania systemu.

4.1.1 Wymagania programowe

Do uruchomienia systemu TuneScript wymagane jest odpowiednie środowisko programowe. Na serwerze konieczny jest system operacyjny Linux (zalecany Ubuntu 20.04 LTS lub nowszy) wraz z zainstalowanym Dockerem w wersji 20.10 lub nowszej oraz Docker Compose w wersji 1.29 lub nowszej. Po stronie klienta wymagana jest jedynie nowoczesna przeglądarka internetowa, taka jak Chrome 90+, Firefox 88+, Safari 14+ lub Edge 90+.

4.2 Kategorie użytkowników

System TuneScript przewiduje trzy główne kategorie użytkowników, różniące się poziomem dostępu i możliwościami w systemie. Niezarejestrowani użytkownicy mogą przeglądać jedynie publiczne transkrypcje. Zarejestrowani użytkownicy otrzymują dostęp do pełnej funkcjonalności systemu, włączając możliwość przysyłania plików audio, generowania transkrypcji oraz zarządzania własnymi materiałami. Administratorzy posiadają pełny dostęp do systemu, w tym do panelu administracyjnego, umożliwiającego zarządzanie użytkownikami i konfiguracją systemu.

4.3 Sposób obsługi

Korzystanie z systemu TuneScript zostało zaprojektowane w sposób intuicyjny i przyjazny dla użytkownika. Poniżej przedstawiono główne funkcjonalności systemu oraz sposób ich obsługi:

1. Rejestracja i logowanie:

- Użytkownik może zarejestrować się, podając nazwę użytkownika, adres e-mail i hasło.
- Następnie otrzymuje e-mail z linkiem potwierdzającym.
- Po potwierdzeniu, użytkownik może zalogować się do systemu.

2. Przesyłanie pliku audio:

- Zalogowany użytkownik może przesłać plik audio w formacie MP3 lub WAV.
- System weryfikuje format pliku.

3. Generowanie transkrypcji:

- Po przesłaniu pliku, system automatycznie rozpoczyna proces transkrypcji.
- Użytkownik jest informowany o postępie procesu.

4. Zarządzanie transkrypcjami:

- Użytkownik może przeglądać, edytować i usuwać swoje transkrypcje.
- Możliwe jest ustawienie widoczności transkrypcji jako publicznej lub prywatnej.

5. Wyszukiwanie i przeglądanie transkrypcji:

- Użytkownicy mogą wyszukiwać transkrypcje po tytule, kompozytorze i widoczności.
- Możliwe jest przeglądanie publicznych transkrypcji innych użytkowników.

6. Ocenianie i dodawanie do ulubionych:

- Zalogowani użytkownicy mogą oceniać transkrypcje.
- Możliwe jest dodawanie transkrypcji do listy ulubionych.

4.4 Administracja systemem

Panel administracyjny systemu TuneScript zapewnia administratorom pełną kontrolę nad funkcjonowaniem aplikacji. Administrator ma możliwość zarządzania kontami użytkowników, moderowania treści oraz monitorowania aktywności systemu. Panel umożliwia również konfigurację kluczowych ustawień systemu i generowanie raportów dotyczących jego wykorzystania. Do głównych zadań administracyjnych należy weryfikacja nowych użytkowników, monitorowanie jakości transkrypcji oraz zarządzanie zasobami systemowymi.

4.5 Kwestie bezpieczeństwa

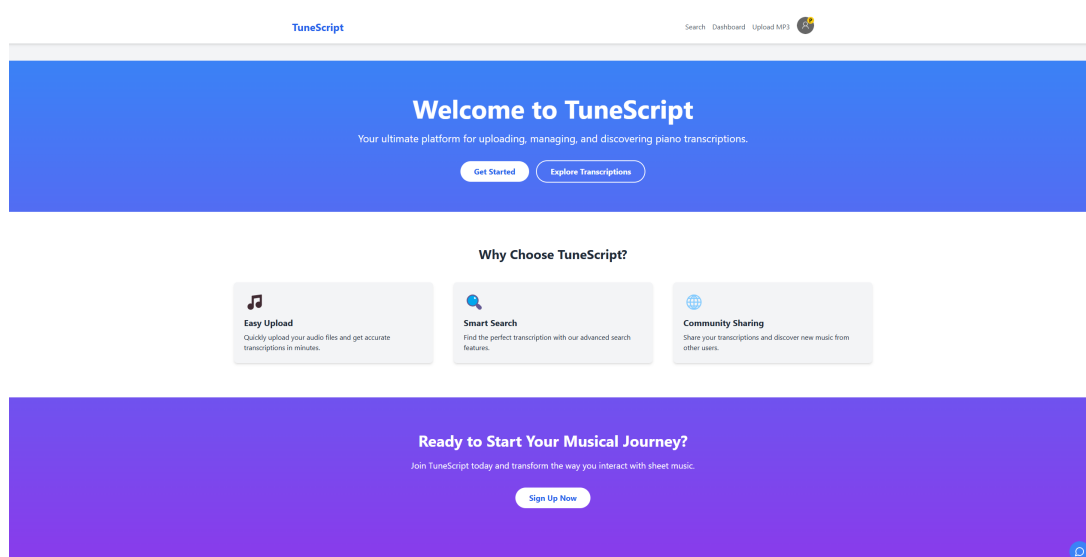
Bezpieczeństwo danych i prywatność użytkowników stanowią kluczowy aspekt systemu TuneScript. Wszystkie dane przesyłane między klientem a serwerem są szyfrowane przy użyciu protokołu HTTPS (ang. Hypertext Transfer Protocol Secure, protokół bezpiecznej komunikacji w sieci), a hasła użytkowników przechowywane są w formie zaszyfrowanej z wykorzystaniem nowoczesnych algorytmów haszujących. System implementuje mechanizm uwierzytelniania oparty na tokenach JWT (ang. JSON Web Token, token uwierzytelniający w formacie JSON) oraz zaawansowane zabezpieczenia przed popularnymi rodzajami ataków, takimi jak CSRF (ang. Cross-Site Request Forgery, fałszowanie żądań międzywitrynowych) i XSS (ang. Cross-Site Scripting, skrypty międzywitrynowe). Regularnie przeprowadzane są aktualizacje wszystkich wykorzystywanych bibliotek i zależności, co pozwala na utrzymanie wysokiego poziomu bezpieczeństwa. System został również wyposażony w mechanizmy monitorowania i wykrywania potencjalnych zagrożeń.

4.6 Przykład działania

Dla lepszego zrozumienia funkcjonowania systemu, poniżej przedstawiono szczegółowy scenariusz użycia aplikacji TuneScript wraz z rzeczywistymi widokami interfejsu użytkownika. Przykład ten ilustruje typową ścieżkę użytkownika od momentu pierwszego kontaktu z systemem do pełnego wykorzystania jego funkcjonalności.

4.6.1 Rozpoczęcie pracy z systemem

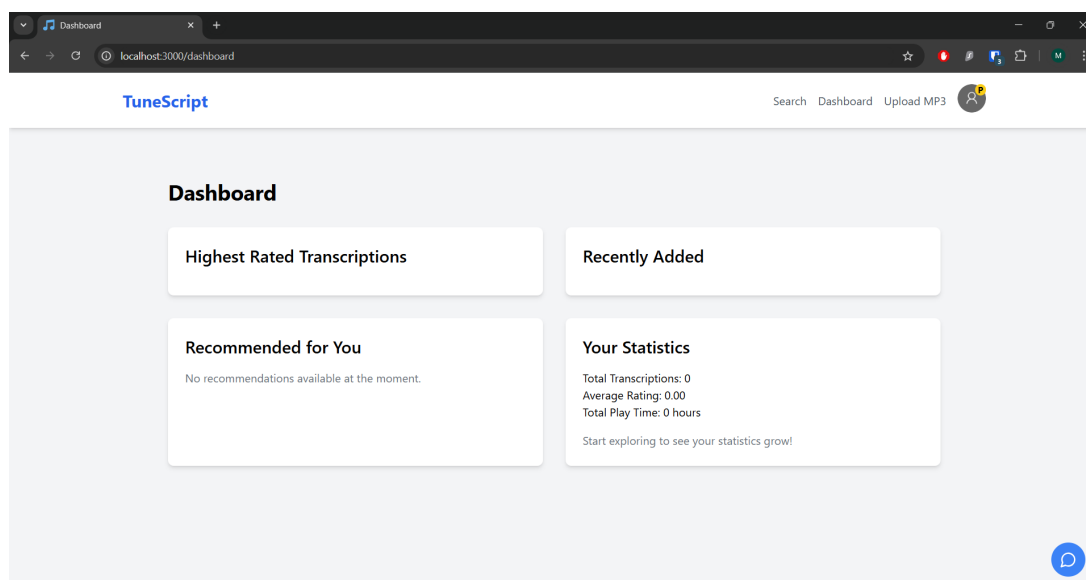
Użytkownik rozpoczyna swoją przygodę z systemem od strony głównej (Rys. 4.1). Interfejs powitalny prezentuje podstawowe informacje o możliwościach systemu oraz oferuje opcje rejestracji i logowania. Strona główna została zaprojektowana w sposób przejrzysty i intuicyjny, umożliwiając szybkie zapoznanie się z głównym przeznaczeniem aplikacji.



Rysunek 4.1: Strona główna systemu TuneScript

4.6.2 Panel główny i podstawowe funkcje

Po pomyślnym zalogowaniu, użytkownik zostaje przekierowany do panelu głównego (Rys. 4.2). Jest to centralne miejsce systemu, prezentujące najważniejsze funkcje i statystyki. Panel zawiera sekcje z najwyżej ocenianymi transkrypcjami, ostatnio dodanymi utworami oraz personalizowanymi rekomendacjami. Użytkownik może stąd szybko przejść do wszystkich głównych funkcji systemu.

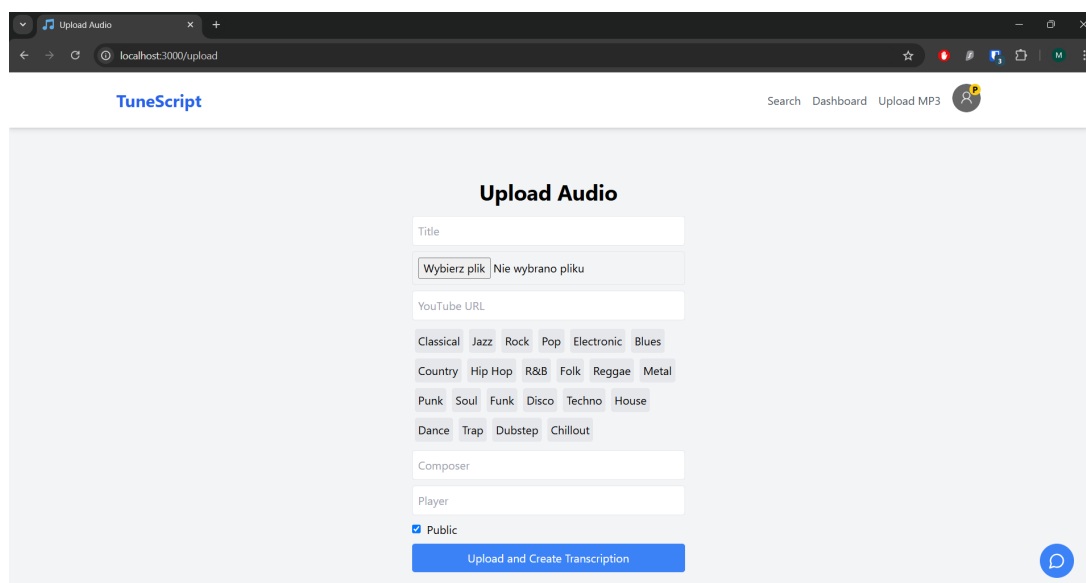


Rysunek 4.2: Panel główny użytkownika

4.6.3 Proces transkrypcji

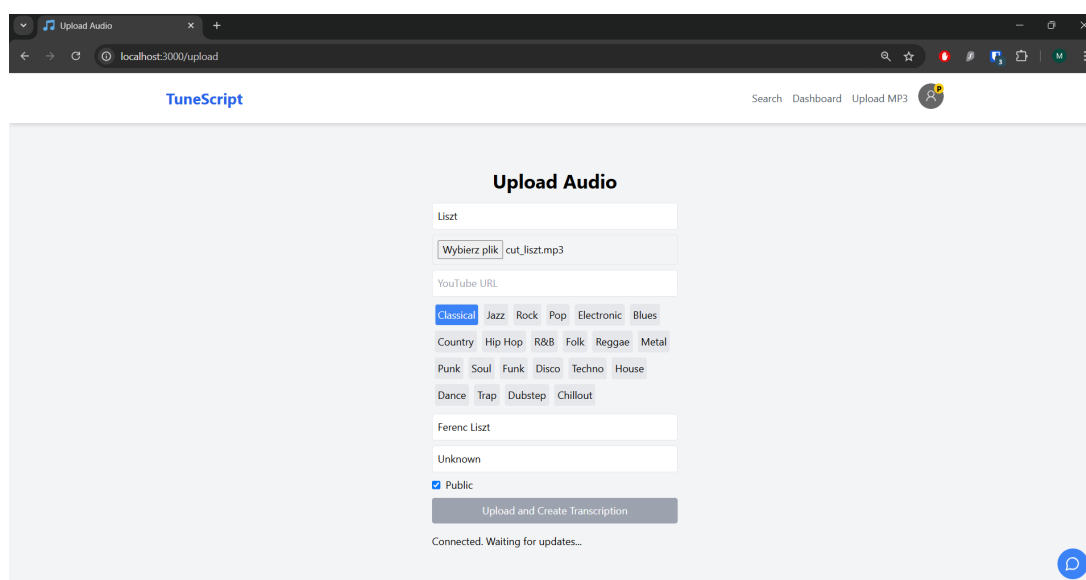
Kluczową funkcjonalnością systemu jest proces transkrypcji plików audio. Użytkownik może rozpocząć ten proces przechodząc do dedykowanego ekranu przesyłania plików (Rys. 4.3). In-

terfejs umożliwia zarówno przesłanie pliku z dysku lokalnego, jak i podanie linku do nagrania w serwisie YouTube.



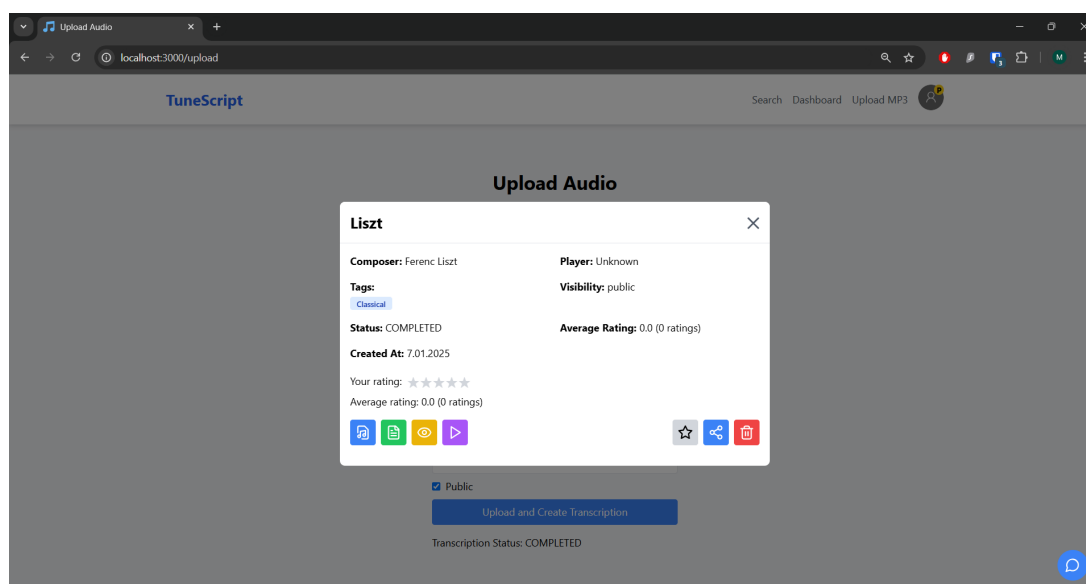
Rysunek 4.3: Ekran przesyłania pliku audio

W trakcie przetwarzania pliku, system wyświetla szczegółowy status postępu transkrypcji (Rys. 4.4), informując o kolejnych etapach procesu.



Rysunek 4.4: Status procesu transkrypcji

Po zakończeniu transkrypcji (Rys. 4.5), system automatycznie generuje podgląd wyników.

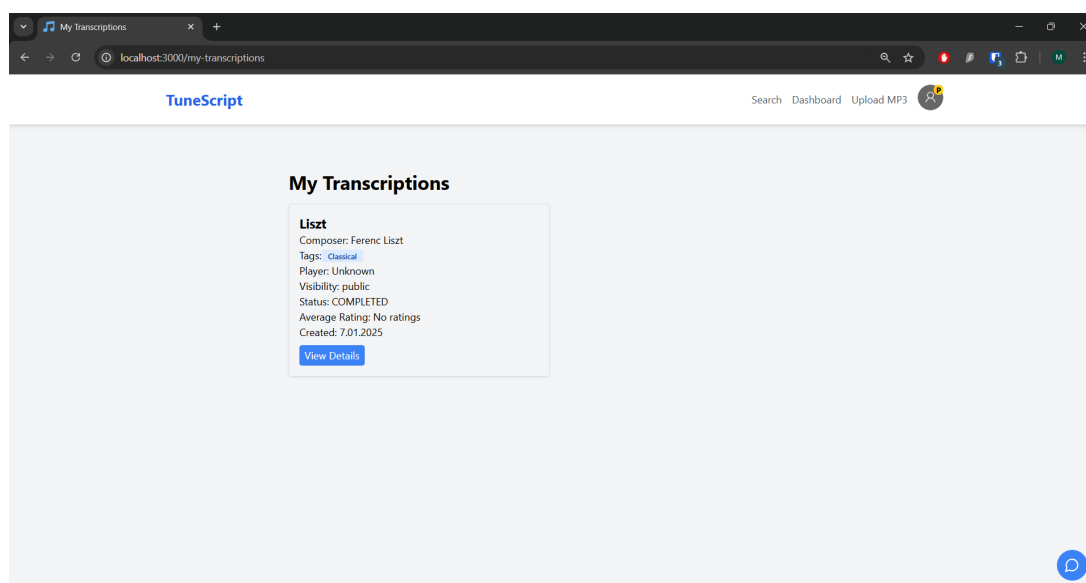


Rysunek 4.5: Zakończenie procesu transkrypcji

Użytkownik otrzymuje dostęp do szczegółowego widoku transkrypcji (Rys. 4.6), gdzie może przeglądać wygenerowane materiały, w tym nuty w formacie PDF oraz plik MIDI. W tym miejscu możliwe jest również odsłuchanie oryginalnego nagrania i porównanie go z wygenerowaną transkrypcją.

4.6.4 Zarządzanie transkrypcjami

Wszystkie transkrypcje użytkownika są dostępne w dedykowanej sekcji "Moje transkrypcje" (Rys. 4.7). Interfejs ten umożliwia łatwe zarządzanie własną biblioteką, włączając w to możliwość edycji metadanych, usuwania transkrypcji czy zmiany ich widoczności.



Rysunek 4.7: Lista transkrypcji użytkownika

Bach Composition

Composer: Johann Sebastian Bach

Player: Unknown

Tags:

PopR&BSoul

Visibility: public

Status: COMPLETED

Average Rating: 3.0 (6 ratings)

Created At: 22.12.2024

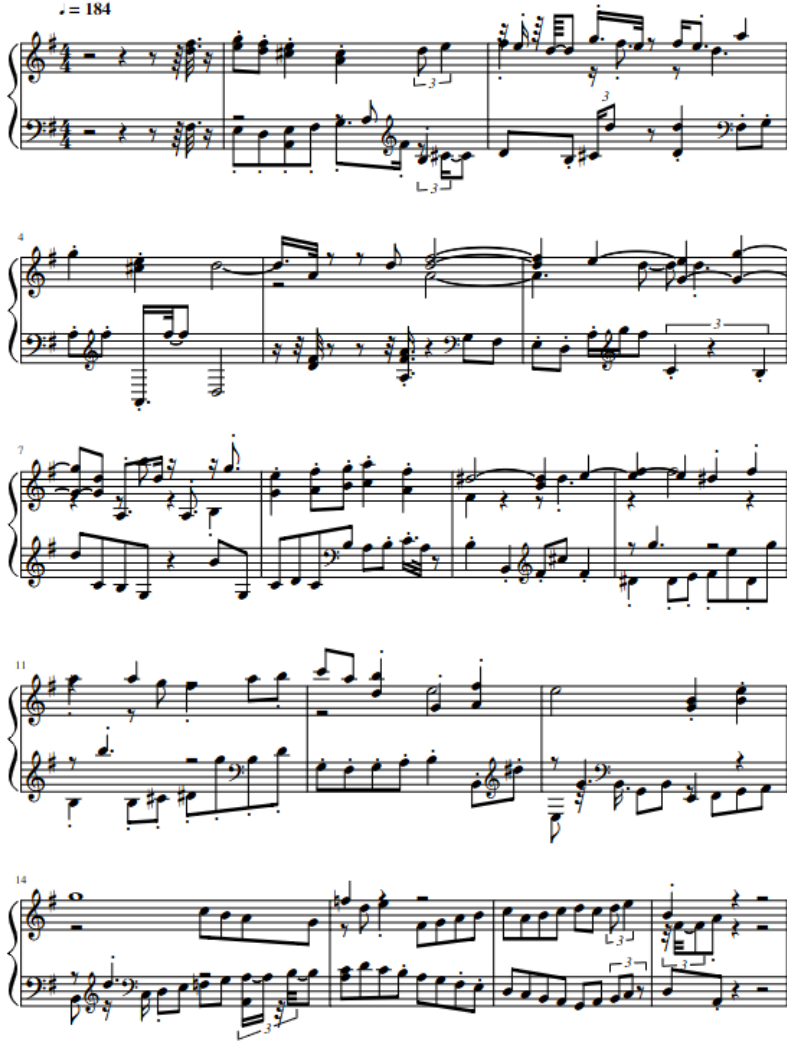
Your rating: ★★★★★

Average rating: 3.0 (6 ratings)



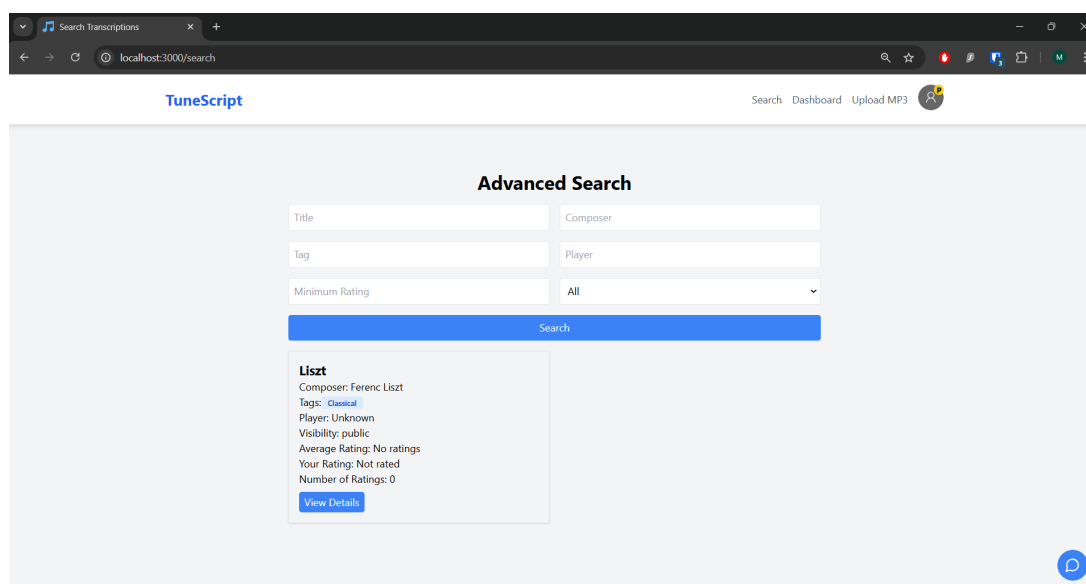


$\text{♩} = 184$



Page 1 of 1

Rysunek 4.6: Szczegółowy widok transkrypcji



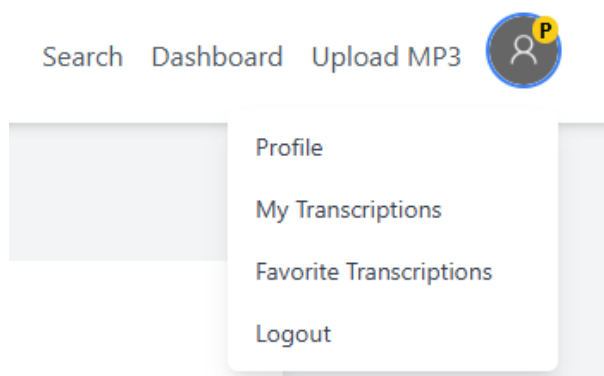
Rysunek 4.8: Wyszukiwanie transkrypcji

System oferuje również zaawansowane możliwości wyszukiwania i filtrowania (Rys. 4.8), co jest szczególnie przydatne przy większej liczbie utworów.

4.6.5 Personalizacja i ustawienia

System zapewnia możliwości personalizacji poprzez panel profilu użytkownika (Rys. 4.9).

Użytkownik może tu dostosować swoje preferencje (Rys. 4.10), zarządzać ustawieniami prywatności oraz modyfikować szczegóły swojego konta.



Rysunek 4.10: Ustawienia preferencji użytkownika

Dodatkowo, istnieje możliwość utworzenia listy ulubionych transkrypcji (Rys. 4.11), co ułatwia szybki dostęp do najczęściej używanych materiałów.

TuneScript

SearchDashboardUpload MP3

Your Profile

Profile Picture



Change Picture

Personal Information

Bio

☐ Make profile public

Preferences

☐ Receive email notifications

☐ Use dark mode

Update Profile

Subscription Status

Active Premium Member

Start Date: 22.12.2024

End Date: 21.01.2025

Cancel Subscription

Change Password

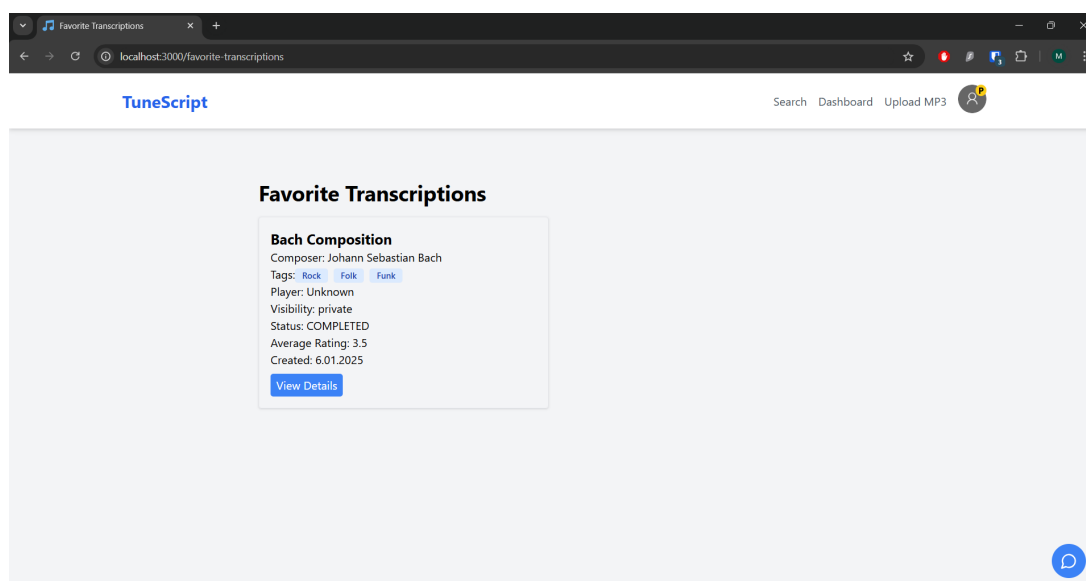
Current Password

New Password

Confirm New Password

Update Password

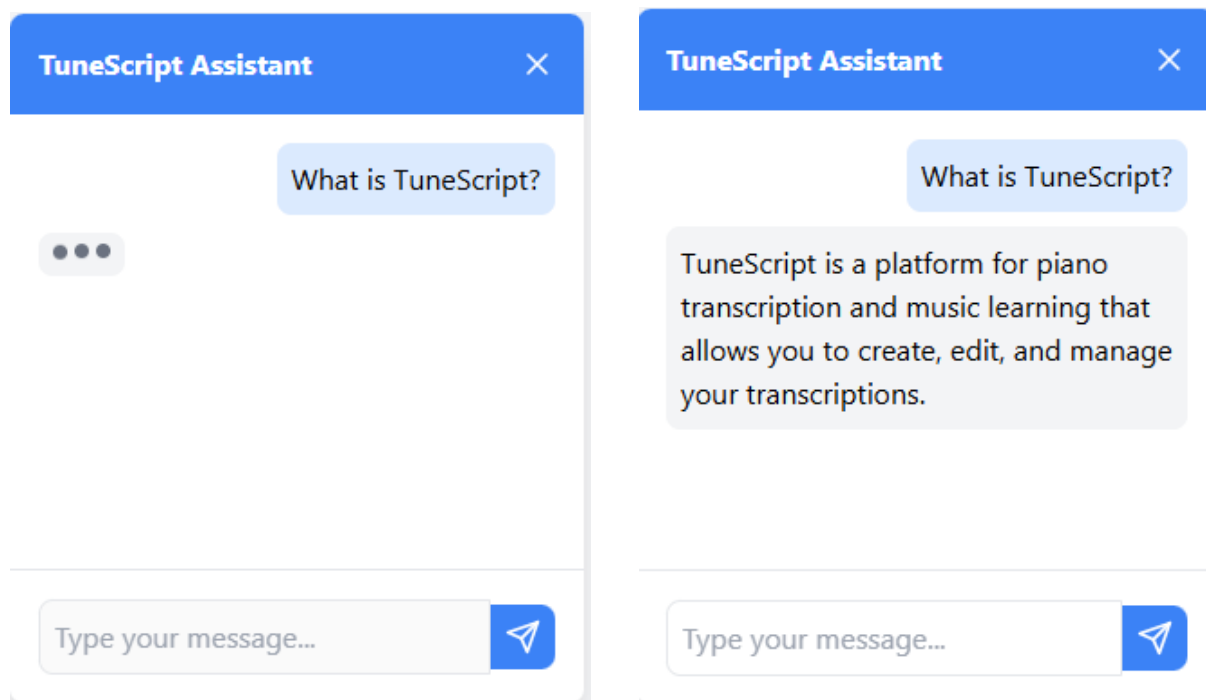
Rysunek 4.9: Profil użytkownika



Rysunek 4.11: Lista ulubionych transkrypcji

4.6.6 Wsparcie użytkownika

Ważnym elementem systemu jest wbudowany asystent chatbot, który służy pomocą w przypadku pytań czy problemów. Asystent oferuje natychmiastową pomoc (Rys. 4.12), odpowiadając na pytania dotyczące funkcjonalności systemu, procesu transkrypcji czy rozwiązywania potencjalnych problemów. Jest to szczególnie przydatne dla nowych użytkowników, którzy dopiero zapoznają się z systemem.



(a) Oczekiwanie na odpowiedź

(b) Odpowiedź

Rysunek 4.12: Asystent chatbot

Rozdział 5

Specyfikacja wewnętrzna

Rozdział ten przedstawia szczegółową specyfikację wewnętrzną systemu TuneScript. Omówione zostaną aspekty architektury systemu, wykorzystane technologie oraz kluczowe decyzje implementacyjne. Szczególna uwaga zostanie poświęcona architekturze mikroservisowej, która została wybrana jako fundament systemu, strukturom danych oraz algorytmom wykorzystanym w procesie transkrypcji muzycznej.

5.1 Przedstawienie idei

TuneScript to zaawansowana aplikacja webowa, która automatyzuje proces transkrypcji nagrań fortepianowych w formacie MP3 na partytury w formacie PDF. Główna idea projektu opiera się na wykorzystaniu sztucznej inteligencji, a dokładniej głębokich sieci neuronowych, do analizy i interpretacji sygnału audio, a następnie przekształcenia go w notację muzyczną.

Kluczowe aspekty idei TuneScript obejmują:

- Wykorzystanie zaawansowanych technik przetwarzania sygnałów audio do precyzyjnej detekcji nut.
- Implementację modelu głębokiego uczenia do transkrypcji muzyki fortepianowej.
- Integrację procesu transkrypcji z systemem generowania partytur.
- Stworzenie przyjaznego dla użytkownika interfejsu webowego do zarządzania transkrypcjami.

5.2 Architektura systemu

TuneScript wykorzystuje architekturę mikroservisową (Microservices Architecture), która umożliwia niezależne rozwijanie, wdrażanie i skalowanie poszczególnych komponentów systemu. Architektura składa się z następujących serwisów:

1. **Serwis frontendowy** jest implementowany w Next.js i TypeScript. Głównym zadaniem tego serwisu jest zapewnienie responsywnego interfejsu użytkownika. Komunikacja z backendem odbywa się poprzez GraphQL API. Serwis ten jest również odpowiedzialny za monitorowanie postępu transkrypcji poprzez SSE.
2. **Serwis backendowy** został zbudowany przy użyciu Django z GraphQL (Graphene). Jest odpowiedzialny za zarządzanie logiką biznesową aplikacji oraz autentykację i autoryzację użytkowników. Serwis koordynuje również proces transkrypcji i generowania partytur.
3. **Serwis pracownika (worker)** bazuje na technologii Celery i jest odpowiedzialny za przetwarzanie zadań asynchronicznych. Jego głównym zadaniem jest wykonywanie transkrypcji plików audio do formatu MIDI oraz generowanie partytur PDF z plików MIDI.
4. **Serwis chatbota** jest implementowany przy użyciu Flask. Wykorzystuje architekturę RAG do obsługi zapytań użytkowników i zapewnia wsparcie w czasie rzeczywistym. Działa jako niezależny serwis pomocniczy, wspierający interakcję z użytkownikiem.

Komponenty infrastrukturalne:

1. Serwisy przechowywania:

- PostgreSQL jako główna baza danych.
- Redis do buforowania danych tymczasowych i kolejkowania zadań.
- System plików do przechowywania plików multimedialnych.

Zastosowana architektura mikroserwisowa zapewnia następujące korzyści:

- **Modularność:** Każdy serwis jest niezależnym komponentem.
- **Skalowalność:** Możliwość skalowania poszczególnych serwisów według potrzeb.
- **Niezawodność:** Awaria jednego serwisu nie wpływa na działanie pozostałych.
- **Elastyczność technologiczna:** Możliwość wyboru optymalnych technologii dla każdego serwisu.
- **Efektywność rozwoju:** Łatwiejsze zarządzanie kodem i wdrożeniami.

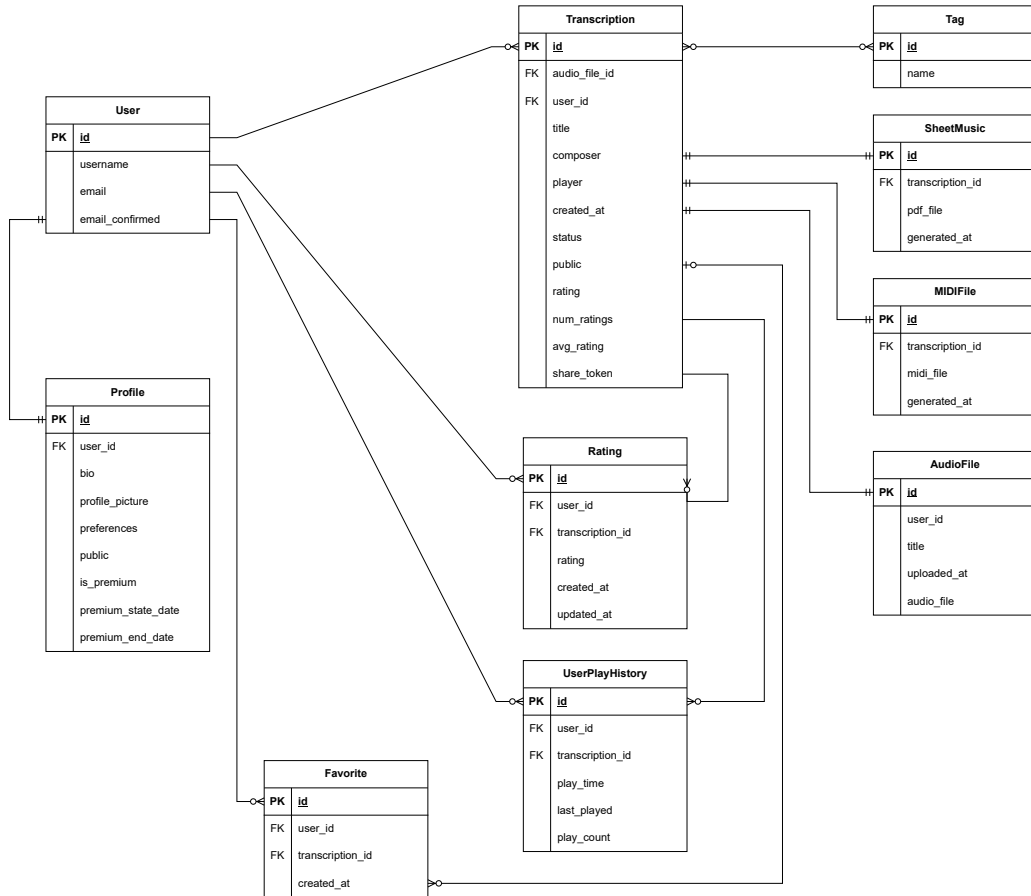
Całość systemu jest uruchamiana w kontenerach przy użyciu systemu *Docker*, a koordynacja współdziałających usług realizowana jest za pomocą narzędzia *Docker Compose*, co zapewnia spójne środowisko zarówno dla rozwoju, jak i wdrożenia.

5.3 Opis struktur danych i organizacji bazy danych

TuneScript wykorzystuje relacyjną bazę danych PostgreSQL do przechowywania danych. Schemat bazy danych przedstawiony jest na Rysunku 5.1. Główne tabele w bazie danych to:

- **User:** Rozszerzenie standardowego modelu użytkownika Django, zawierające dodatkowe pole do potwierdzenia e-maila.
- **Profile:** Przechowuje dodatkowe informacje o użytkowniku, takie jak biografia, zdjęcie profilowe, preferencje oraz informacje o statusie premium.
- **AudioFile:** Zawiera informacje o przesłanych plikach audio, w tym tytuł i ścieżkę do pliku.
- **Transcription:** Przechowuje metadane transkrypcji, w tym odniesienia do pliku audio, użytkownika, tagów, oraz informacje o statusie, ocenach i widoczności.
- **Rating:** Zawiera oceny użytkowników dla poszczególnych transkrypcji.
- **Favorite:** Przechowuje informacje o ulubionych transkrypcjach użytkowników.
- **MIDIFile:** Zawiera informacje o wygenerowanych plikach MIDI dla transkrypcji.
- **SheetMusic:** Przechowuje informacje o wygenerowanych partyturach PDF dla transkrypcji.
- **Tag:** Zawiera tagi, które mogą być przypisane do transkrypcji.
- **UserPlayHistory:** Przechowuje historię odtwarzania transkrypcji przez użytkowników, w tym czas odtwarzania i liczbę odtworzeń.

Tabele te są ze sobą powiązane za pomocą kluczy obcych, tworząc kompleksową strukturę danych, która umożliwia efektywne zarządzanie informacjami w systemie TuneScript. Jak widać na Rysunku 5.1, szczególną uwagę zwrócono na optymalizację wydajności poprzez zastosowanie indeksów oraz unikalnych ograniczeń tam, gdzie jest to konieczne.



Rysunek 5.1: Schemat bazy danych TuneScript

5.4 Skrypty deweloperskie

Projekt TuneScript zawiera skrypty ułatwiające proces rozwoju i testowania. Skrypty te, pokazane w listingu 5.1, obejmują między innymi uruchamianie i zatrzymywanie kontenerów Docker, testowanie aplikacji oraz formatowanie kodu.

5.5 Szczegóły implementacji wybranych fragmentów

W tej sekcji przedstawiono szczegółową implementację kluczowych elementów systemu TuneScript. Szczególną uwagę poświęcono implementacji algorytmu transkrypcji przedstawionego w rozdziale 2, a także systemowi kolejkowania zadań oraz mechanizmowi monitorowania postępu transkrypcji w czasie rzeczywistym. Wszystkie fragmenty kodu znajdują się w dodatku 8.4.

5.5.1 Implementacja modelu transkrypcji

Przedstawiony w rozdziale 2 algorytm transkrypcji został zaimplementowany z wykorzystaniem zaawansowanej architektury konwolucyjno-rekurencyjnej. Poniżej przedstawiono szczegóły techniczne każdego z etapów algorytmu:

Listing 5.1: Skrypty deweloperskie

```

1  {
2      "name": "tunescript-devtools",
3      "version": "1.0.0",
4      "description": "Development tools for TuneScript",
5      "scripts": {
6          "start": "docker-compose up -d",
7          "stop": "docker-compose down -v",
8          "start:persistent": "docker-compose -f docker-compose.
9              persistent.yml up -d",
10         "stop:persistent": "docker-compose -f docker-compose.
11             persistent.yml down",
12         "start:fresh": "docker-compose -f docker-compose.fresh.
13             yaml up -d",
14         "stop:fresh": "docker-compose -f docker-compose.fresh.yml
15             down -v",
16         "restart": "npm run stop && npm run start",
17         "restart:persistent": "npm run stop:persistent && npm run
18             start:persistent",
19         "restart:fresh": "npm run stop:fresh && npm run start:
20             fresh",
21         "logs": "docker-compose logs -f",
22         "test": "docker-compose exec frontend npm run test",
23         "test:watch": "docker-compose exec frontend npm run test:
24             watch",
25         "test:coverage": "docker-compose exec frontend npm run
26             test:coverage",
27         "test:backend": "docker-compose exec web poetry run
28             pytest tunescript_app/tests",
29         "format": "docker-compose exec web /app/format.sh"
30     }
31 }

```

1. **Przetwarzanie wstępne sygnału audio:** Implementacja tego etapu obejmuje ekstraktor spektrogramu oraz ekstraktor cech mel-spektrogramu. Jak pokazano w listingu 1, wykorzystano okno Hann'a o rozmiarze 2048 próbek oraz 229 filtrów melowych, co zapewnia optymalną rozdzielczość czasowo-częstotliwościową dla dźwięków fortepianowych.
2. **Detekcja początków i końców nut:** Implementacja wykorzystuje cztery równoległe sieci konwolucyjno-rekurencyjne, każda specjalizująca się w innym aspekcie detekcji: ramek czasowych, początków nut, końców nut oraz głośności. Ta architektura zapewnia precyzyjne wykrywanie poszczególnych parametrów dźwięku.
3. **Przetwarzanie kontekstu czasowego:** Dwukierunkowa sieć rekurencyjna zbudowana w oparciu o komórki GRU (ang. *Gated Recurrent Unit*), zaimplementowana z rozmiarem warstwy ukrytej 256 neuronów, przetwarza sekwencyjnie otrzymane cechy, uwzględniając szerszy kontekst czasowy wykonania utworu.

5.5.2 Implementacja systemu kolejowania transkrypcji

System kolejowania zadań został zaprojektowany do obsługi długotrwałego procesu transkrypcji w sposób asynchroniczny. Jak pokazano w listingu 2, proces transkrypcji jest realizowany jako zadanie Celery z maksymalnie trzema próbami ponownego wykonania w przypadku błędu. Proces rozpoczyna się od pobrania i przygotowania pliku audio do analizy, włączając konwersję do formatu mono. Następnie wykonywana jest właściwa transkrypcja przy użyciu modelu uczenia maszynowego, którego wyniki są zapisywane w pliku tymczasowym w formacie MIDI. W kolejnym kroku system generuje partytury w formacie PDF na podstawie utworzonego pliku MIDI. Status zadania jest na bieżąco aktualizowany w bazie danych, co umożliwi monitorowanie postępu transkrypcji. W przypadku wystąpienia błędu, mechanizm ponownych prób zwiększa niezawodność systemu, próbując wykonać zadanie ponownie po 60 sekundach.

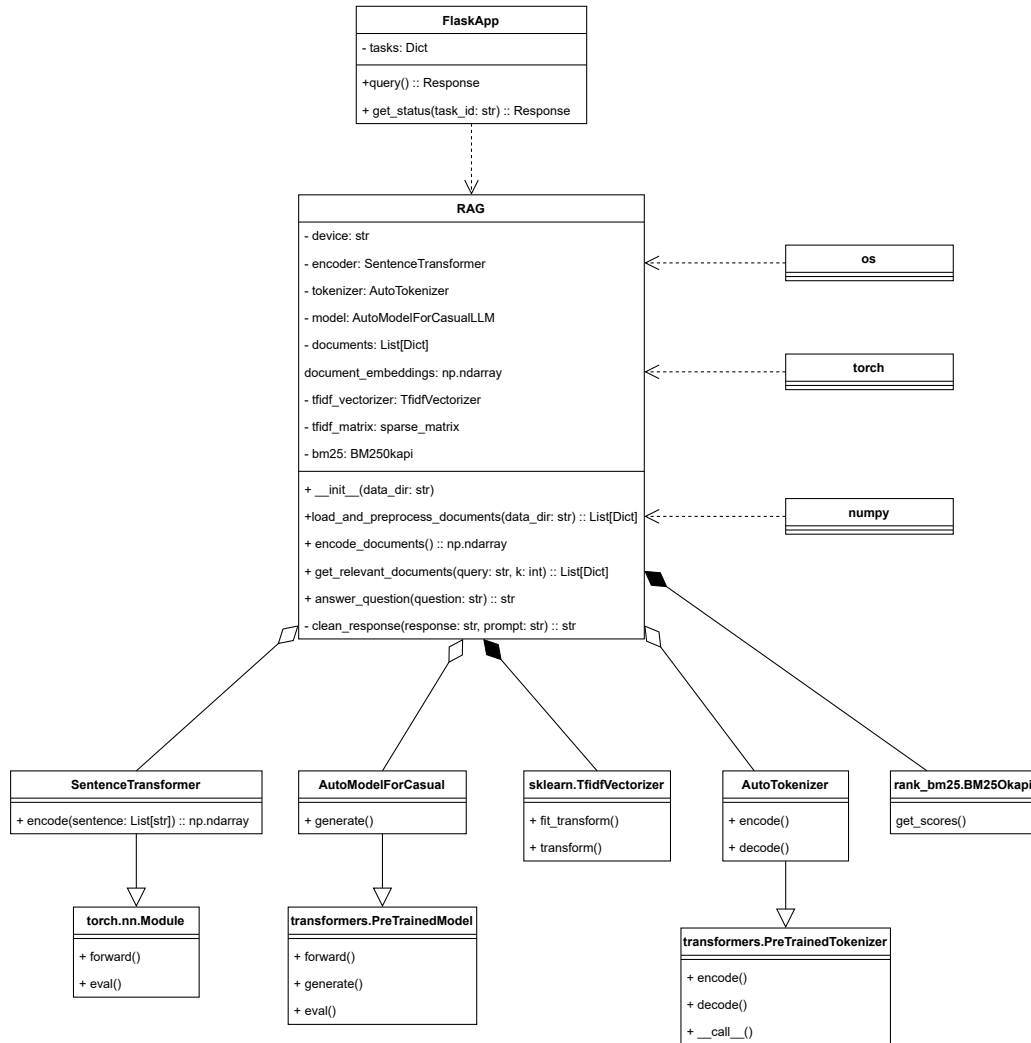
5.5.3 Implementacja systemu SSE do monitorowania postępu

Mechanizm SSE (zdarzenia wysyłane przez serwer) umożliwia przekazywanie informacji o postępie transkrypcji do interfejsu użytkownika w czasie rzeczywistym. Implementacja tego mechanizmu, przedstawiona w listingu 3, obejmuje endpoint HTTP, który ustanawia długotrwałe połączenie z przeglądarką klienta. W regularnych odstępach czasu, co 2 sekundy, system sprawdza status transkrypcji w bazie danych i przysyła do klienta aktualne informacje o statusie transkrypcji, tytule utworu oraz URL-ach do wygenerowanych plików MIDI i PDF, jeśli są dostępne. Implementacja zawiera również mechanizmy obsługi sytuacji wyjątkowych, takich jak nieistniejące transkrypcje czy przerwane połączenia. Połączenie jest automatycznie zamykane po zakończeniu transkrypcji (status ‘COMPLETED’) lub w przypadku wystąpienia błędu (status ‘FAILED’).

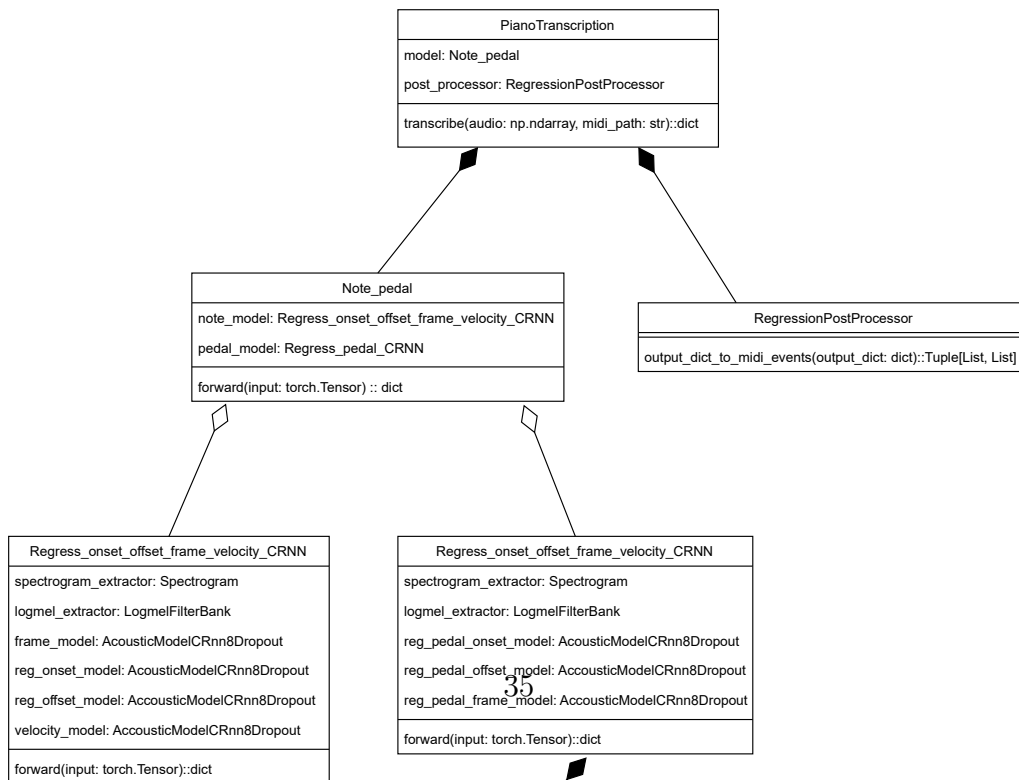
5.6 Diagramy UML

W celu lepszego zobrazowania struktury systemu i zależności między jego komponentami, poniżej przedstawiono diagramy klas UML dla kluczowych modułów systemu. Szczególną uwagę poświęcono modelowi generacji ze wspomaganie wyszukiwania (RAG, ang. *Retrieval-Augmented Generation*) wykorzystywanemu w chatbocie (rys. 5.2) oraz implementacji modelu sztucznej inteligencji odpowiedzialnego za transkrypcję muzyczną (rys. 5.3).

Diagram przedstawiony na rysunku 5.2 pokazuje strukturę modułu RAG wykorzystywanego w chatbocie. System ten łączy w sobie komponenty odpowiedzialne za przetwarzanie zapytań użytkowników, wyszukiwanie odpowiedniej dokumentacji oraz generowanie odpowiedzi z wykorzystaniem modelu językowego.



Rysunek 5.2: Diagram klas modułu RAG



Na rysunku 5.3 zaprezentowano strukturę modułu odpowiedzialnego za transkrypcję muzyczną. Widoczne są klasy implementujące model głębokiego uczenia, przetwarzanie sygnału audio oraz generowanie partytur. System ten wykorzystuje zaawansowane techniki uczenia maszynowego do precyzyjnej detekcji nut i ich parametrów.

Rozdział 6

Model transkrypcji

Implementacja modelu do transkrypcji została oparta na pracy Kong et al. (2020) [15], która przedstawia wysokorozdzielczościowy system transkrypcji fortepianu. System ten wyróżnia się możliwością precyzyjnego określania czasów początku i końca dźwięków.

6.1 Architektura modelu

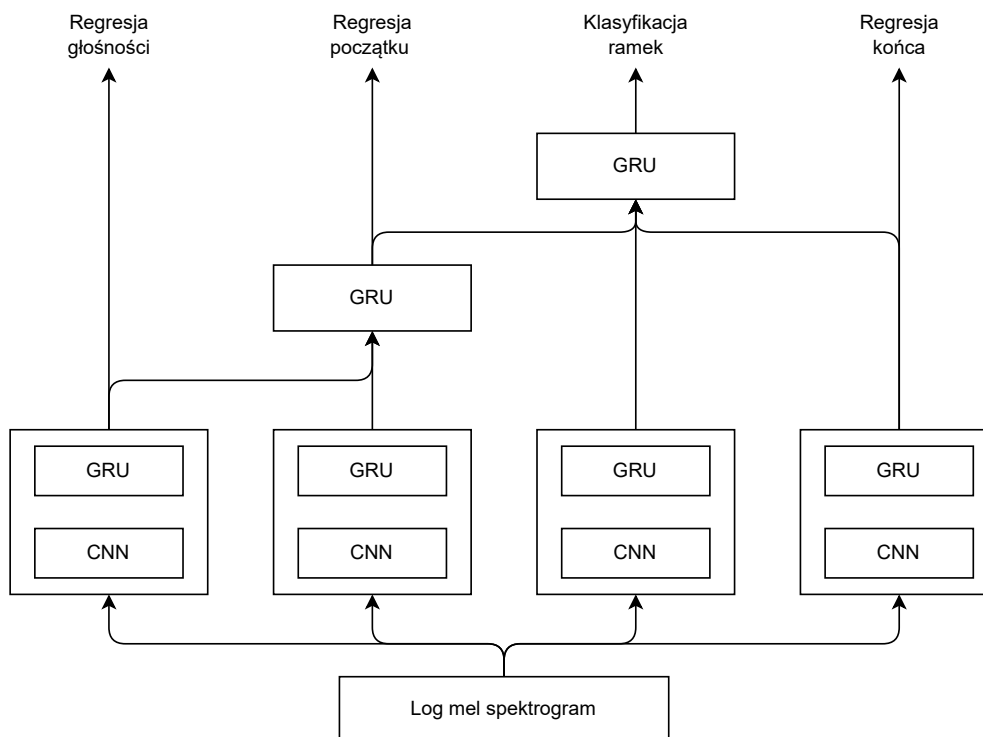
Model transkrypcji dźwięku został zaprojektowany jako struktura zawierająca cztery główne podmoduły: moduł regresji głośności dźwięku, moduł regresji początku dźwięku, moduł klasyfikacji ramek czasowych oraz moduł regresji końca dźwięku. Każdy z tych podmodułów jest oparty na wspólnej architekturze akustycznej, która wykorzystuje kombinację warstw konwolucyjnych (CNN) i rekurencyjnych (biGRU).

Architektura pojedynczego modelu akustycznego składa się z kilku kluczowych elementów. Na początku znajduje się cztery bloki konwolucyjne, z których każdy zwiększa liczbę kanałów wyjściowych w kolejnych krokach: 48, 64, 96 i 128. Każdy blok konwolucyjny obejmuje dwie warstwy konwolucyjne o wymiarach 3x3, po których następuje normalizacja wsadowa oraz funkcja aktywacji ReLU. Dodatkowo każdy blok redukuje wymiarowość danych w dziedzinie częstotliwości o połowę.

Następnie dane przechodzą przez warstwę w pełni połączoną zawierającą 768 jednostek, a potem przez dwie dwukierunkowe warstwy GRU, z których każda ma 256 jednostek ukrytych. Na końcu każda ścieżka kończy się warstwą w pełni połączoną z funkcją aktywacji sigmoid, umożliwiającą odpowiednią interpretację wyników dla zadań regresji i klasyfikacji.

Cztery główne podmoduły korzystają z opisanej architektury w kontekście różnych celów: - Moduł regresji głośności wykorzystuje informacje do przewidywania intensywności dźwięku. - Moduł regresji początku dźwięku przewiduje momenty inicjacji dźwięków. - Moduł klasyfikacji ramek czasowych określa, które ramki należą do danego dźwięku. - Moduł regresji końca dźwięku odpowiada za przewidywanie momentu zakończenia dźwięków.

Cała architektura została zaprojektowana tak, aby skutecznie wykorzystywać dane wejściowe w postaci spektrogramu melowego, jak przedstawiono na rysunku 6.1.



Rysunek 6.1: Diagram architektury modelu transkrypcji

6.2 Proces uczenia

Proces uczenia modelu obejmuje kilka kluczowych etapów, począwszy od odpowiedniego przetworzenia danych wejściowych, przez konfigurację parametrów treningu, aż po zdefiniowanie funkcji straty. Każdy z tych elementów został starannie dobrany w celu osiągnięcia optymalnej jakości transkrypcji.

6.2.1 Przetwarzanie wstępne danych

Do treningu wykorzystano zbiór MAESTRO V2.0.0 [11], zawierający ponad 200 godzin nagrań fortepianowych pochodzących z Międzynarodowego Konkursu Pianistycznego e-Competition. Nagrania zostały wykonane na fortepianach koncertowych Yamaha Disklavier wyposażonych w precyzyjny system rejestracji i odtwarzania MIDI. W ramach przetwarzania wstępnego nagrania zostały przekonwertowane do postaci monofonicznej oraz przepróbkowane do częstotliwości 16 kHz. Następnie dane zostały podzielone na 10-sekundowe fragmenty, które były następnie przekształcane do postaci mel-spektrogramu. W procesie tym wykorzystano okno Hannu o rozmiarze 2048 próbek, stosując 229 filtrów melowych w zakresie częstotliwości od 30 Hz do 8000 Hz. Krok przesunięcia okna został ustalony na 10 ms, co zapewnia odpowiednią rozdzielczość czasową dla analizy dźwięku fortepianowego.

6.2.2 Parametry treningu

Proces uczenia modelu został skonfigurowany z wykorzystaniem optymalizatora Adam, dla którego przyjęto początkowy współczynnik uczenia równy 0.0005. W trakcie treningu współczynnik ten był redukowany o mnożnik 0.9 co 10000 iteracji, co pozwoliło na stopniowe dostrajanie parametrów modelu. Trening przeprowadzono z rozmiarem wsadu równym 12, przez 200000 iteracji, co zajęło około 4 dni na pojedynczej karcie graficznej Tesla-V100-PCIE-32GB. W celu zapobiegania przeuczeniu zastosowano regularyzację typu dropout, przyjmując współczynnik 0.2 dla warstw po blokach konwolucyjnych oraz 0.5 dla warstw w pełni połączonych. Wszystkie hiperparametry zostały zoptymalizowane na zbiorze walidacyjnym.

6.2.3 Funkcja straty

Całkowita funkcja straty jest sumą czterech składowych:

$$l_{note} = l_{fr} + l_{bp} + l_{kd} + l_g \quad (6.1)$$

gdzie:

- l_{fr} - strata ramek czasowych
- l_{bp} - strata początku dźwięku
- l_{kd} - strata końca dźwięku
- l_g - strata głośności

Wszystkie składowe wykorzystują entropię krzyżową jako funkcję straty bazową. Składowe funkcje straty są traktowane z równymi wagami, co okazało się skuteczne w eksperymentach.

6.3 Wnioskowanie

Podczas wnioskowania model przetwarza mel-spektrogram nagrania i generuje prawdopodobieństwa dla każdej ramki czasowej. Wykrywanie precyzyjnych czasów początku i końca dźwięku wykorzystuje algorytm analityczny bazujący na lokalnych maksimach w przewidywaniach modelu. Progi decyzyjne zostały ustalone eksperymentalnie na poziomie 0.3 dla wszystkich typów detekcji (początku dźwięku, końca dźwięku, ramki).

System osiąga wysoką skuteczność w wykrywaniu początków nut, osiągając miarę F1 na poziomie 96.72% (przy precyzji 98.17% i czułości 95.35%). W bardziej złożonym zadaniu wykrywania zarówno początków jak i końców nut (offset) skuteczność wynosi 82.47% (precyzja 83.68%, czułość 81.32%). Przy uwzględnieniu dodatkowo dynamiki (velocity) system osiąga wynik F1 równy 80.92% (precyzja 82.10%, czułość 79.80%). Wyniki te stanowią potwierdzenie skuteczności zastosowanej architektury.

Rozdział 7

Weryfikacja i walidacja

Proces weryfikacji i walidacji jest kluczowym elementem w tworzeniu oprogramowania, zapewniającym jego niezawodność i zgodność z wymaganiami. W przypadku systemu TuneScript, ze względu na złożoność aplikacji i różnorodność funkcjonalności, zastosowano kompleksowe podejście do testowania. Szczególną uwagę poświęcono testowaniu komponentów związanych z przetwarzaniem audio oraz interfejsu użytkownika, które stanowią krytyczne elementy systemu.

7.1 Organizacja testów

System TuneScript wykorzystuje kompleksowe podejście do testowania, bazowane na modelu V [16], gdzie testowanie odbywa się na różnych poziomach abstrakcji. W systemie zaimplementowano testy jednostkowe poszczególnych komponentów oraz testy integracyjne weryfikujące współdziałanie elementów systemu. Szczególny nacisk położono na automatyzację procesu testowego, co pozwala na szybkie wykrywanie potencjalnych problemów w trakcie rozwoju aplikacji.

7.1.1 Testy jednostkowe

Testy jednostkowe stanowią fundament procesu testowego w systemie TuneScript. Ich głównym celem jest weryfikacja poprawności działania pojedynczych komponentów w izolacji od reszty systemu. Aplikacja posiada łącznie 174 testy jednostkowe, w tym 114 testów dla części frontendowej oraz 60 testów dla części backendowej, obejmujących testy modeli, mutacji, zapytań i widoków. Testy te zapewniają, że każdy element systemu działa zgodnie z założeniami.

7.1.2 Testy integracyjne

Testy integracyjne weryfikują poprawność współdziałania poszczególnych komponentów systemu. Jest to szczególnie istotne w kontekście aplikacji webowej, gdzie wiele elementów musi ze sobą współpracować w sposób bezproblemowy. System zawiera zestaw 10 testów integracyjnych, obejmujących wszystkie kluczowe przepływy danych, ze szczególnym uwzględnieniem komunikacji między frontendem a backendem, obsługi autoryzacji oraz przetwarzania strumieni audio.

7.2 Przypadki testowe i wykryte błędy

W ramach procesu rozwoju systemu przeprowadzono szereg testów mających na celu weryfikację poprawności działania poszczególnych komponentów oraz wykrycie potencjalnych błędów. Testy obejmowały zarówno automatyczną weryfikację kodu poprzez testy jednostkowe i integracyjne, jak również manualne testowanie funkcjonalności z perspektywy użytkownika końcowego. Proces testowania pozwolił na wykrycie i naprawę wielu istotnych błędów, przyczyniając się do zwiększenia niezawodności systemu.

7.2.1 Testy automatyczne

Testy automatyczne stanowią fundament procesu weryfikacji jakości kodu w systemie. Wykorzystano framework Jest [17] wraz z biblioteką React Testing Library [30] oraz PyTest [23] do tworzenia i wykonywania testów jednostkowych. Dzięki automatyzacji testów możliwe było szybkie wykrywanie błędów wprowadzanych podczas rozwoju aplikacji.

Testy interfejsu użytkownika

W ramach testów automatycznych interfejsu użytkownika wykryto i naprawiono szereg istotnych błędów. Kluczowym problemem było nieprawidłowe przywracanie stanu autoryzacji użytkownika po odświeżeniu strony, co mogło prowadzić do utraty sesji użytkownika. Test przedstawiony w listingu 7.1 pozwolił zidentyfikować i naprawić ten problem poprzez implementację poprawnej obsługi tokena autoryzacji w pamięci lokalnej przeglądarki.

Listing 7.1: Test przywracania stanu autoryzacji

```
1 it('restores authentication state from localStorage', async () => {
2   mockLocalStorage.getItem.mockImplementation((key) => {
3     if (key === 'token') return 'saved-token';
4     if (key === 'user') return JSON.stringify({
5       id: '2',
6       username: 'saveduser'
7     });
8     return null;
9   });
10
11   render(<AuthProvider><TestComponent /></AuthProvider>);
12
13   await waitFor(() => {
14     expect(screen.getByTestId('auth-status'))
15       .toHaveTextContent('Authenticated');
16     expect(screen.getByTestId('user-info'))
17       .toHaveTextContent('saveduser');
18   });
19 });
```

Testy komunikacji API

Przeprowadzono szereg testów weryfikujących poprawność działania interfejsu API, szczególnie w kontekście strumieniowania danych podczas procesu transkrypcji. Test przedstawiony w listingu 7.2 pozwolił wykryć problemy z obsługą długotrwałych połączeń SSE oraz błędy w obsłudze przerwanych połączeń.

Listing 7.2: Test strumienia SSE

```

1 it('should handle successful SSE stream', async () => {
2   const handlerPromise = handler(mockReq as NextApiRequest, mockRes as
    NextApiResponse);
3
4   // Symulacja strumienia SSE
5   mockStream.pushData('data: {"status":"PENDING"}\n\n');
6   await new Promise(resolve => setTimeout(resolve, 10));
7   mockStream.pushData('data: {"status":"COMPLETED"}\n\n');
8   mockStream.close();
9
10  await handlerPromise;
11
12  expect(mockRes.writeHead).toHaveBeenCalledWith(200, {
13    'Content-Type': 'text/event-stream',
14    'Cache-Control': 'no-cache',
15    'Connection': 'keep-alive',
16  });
17
18  expect(mockRes.write).toHaveBeenCalledTimes(2);
19  expect(mockRes.write).toHaveBeenNthCalledWith(
20    1,
21    'data: {"status":"PENDING"}\n\n'
22  );
23  expect(mockRes.write).toHaveBeenNthCalledWith(
24    2,
25    'data: {"status":"COMPLETED"}\n\n'
26  );
27  expect(mockRes.end).toHaveBeenCalledTimes(1);
28 });

```

Testy endpointów

Istotnym elementem były również testy różnych endpointów API, w szczególności tych związanych z obsługą transkrypcji. Test pokazany w listingu 7.3 ujawnił problemy z obsługą równoległych żądań transkrypcji oraz zarządzaniem stanem podczas długich operacji przetwarzania.

Testy te pozwoliły wykryć i naprawić następujące problemy:

- Nieprawidłową obsługę przerwanych połączeń SSE.
- Problemy z timeoutami podczas długotrwałych transkrypcji.
- Błędy w serializacji odpowiedzi JSON.
- Nieoptymalne zarządzanie zasobami przy wielu równoległych połączeniach.

Listing 7.3: Test endpointu transkrypcji

```

1 it('handles concurrent transcription uploads correctly', async () => {
2   const mockStream = new MockReadableStream();
3   const mockUseQuery = jest.requireMock('@apollo/client').useQuery;
4
5   mockUseQuery.mockReturnValue({
6     loading: false,
7     data: { transcriptions: [] }
8   });
9
10  render(<Upload />);
11
12  // Symulacja równoległego przesyłania plików
13  const audioFiles = [
14    new File(['audio1'], 'song1.mp3', { type: 'audio/mpeg' }),
15    new File(['audio2'], 'song2.mp3', { type: 'audio/mpeg' })
16  ];
17
18  audioFiles.forEach(file => {
19    const fileInput = screen.getByLabelText(/audio file/i);
20    fireEvent.change(fileInput, { target: { files: [file] } });
21    fireEvent.click(screen.getByText(/upload/i));
22  });
23
24  // Weryfikacja kolejkwania zadan
25  await waitFor(() => {
26    expect(mockStream.pushData)
27      .toHaveBeenCalledWith('data: {"status":"PENDING"}\n\n');
28    expect(screen.getAllByText(/processing/i)).toHaveLength(2);
29  });
30 });

```

7.2.2 Testy manualne

Poza testami automatycznymi przeprowadzono szereg testów manualnych, które pozwoliły na weryfikację systemu w rzeczywistych warunkach użytkowania. Szczególną uwagę poświęcono funkcjonalnościom wymagającym interakcji użytkownika oraz długotrwałym procesom przetwarzania danych.

Test importu z YouTube

W ramach testów przeprowadzono serię importów utworów fortepianowych z YouTube. Proces testowy obejmował:

- Import 10-minutowej sonaty fortepianowej Beethovena.
- Weryfikację jakości pobranego dźwięku.
- Monitorowanie postępu transkrypcji.

Testy ujawniły problem z obsługą długich utworów, gdzie system przerywał proces transkrypcji po przekroczeniu domyślnego limitu czasu przetwarzania. Problem rozwiązano poprzez modyfikację konfiguracji Celery i wprowadzenie mechanizmu wznowiania przerwanych zadań.

Test zmiany zdjęcia profilowego

Wprowadzono testy mające na celu weryfikację poprawności zmiany zdjęcia profilowego. Proces testowy obejmował:

- Próby przesłania zdjęcia przez użytkownika.
- Weryfikację poprawności wyświetlania przesłanego zdjęcia w profilu użytkownika.
- Sprawdzenie konfiguracji plików statycznych i ścieżek w środowisku Docker.

Testy ujawniły problem, w którym niepoprawnie załadowane zdjęcia pozostawały na domyślnym obrazie. Problem został rozwiązany poprzez modyfikację konfiguracji plików statycznych i ścieżek w Dockerze.

Test wyświetlania podglądu transkrypcji

W ramach testów zweryfikowano poprawność wyświetlania podglądu transkrypcji:

- Sprawdzone renderowanie podglądu dla różnych przykładów transkrypcji.
- Zidentyfikowano problem z niepoprawnym wyświetlaniem podglądu transkrypcji.

Po wprowadzeniu niezbędnych modyfikacji w komponencie odpowiedzialnym za renderowanie, przeprowadzono kolejną iterację testów, która potwierdziła prawidłowe działanie podglądu transkrypcji dla wszystkich testowanych przypadków.

7.3 Testowanie wydajności

W ramach testów wydajnościowych przeprowadzono serię pomiarów czasu przetwarzania różnych plików audio. Testy obejmowały pliki o różnej długości (od 20 sekund do prawie 9 minut) oraz w różnych formatach (MP3 i WAV). Wyniki przedstawiono w tabeli 7.1.

Analiza wyników prowadzi do następujących wniosków:

- Czas przetwarzania jest proporcjonalny do długości utworu.
- System zachowuje stabilność przy dłuższych plikach.
- Narzut czasowy na przetwarzanie wynosi średnio 10-15% długości utworu.
- Format pliku (MP3 vs WAV) nie ma znaczącego wpływu na czas przetwarzania.

Tabela 7.1: Wyniki testów przetwarzania plików

Plik	Długość utworu	Długość przetwarzania
bach.mp3	00:20	00:21
liszt.mp3	00:40	00:42
C'est la vie.mp3	00:49	01:07
Fletcher's song.wav	01:39	02:10
Intermezz.wav	04:48	05:06
Misty.wav	08:42	09:20

7.4 Najważniejsze pokrycie testami

Kompleksowa analiza pokrycia testami wykazała, że system TuneScript posiada wysoki poziom zabezpieczenia kodu testami. Testy jednostkowe pokrywają:

- 100% komponentów React,
- 100% logiki biznesowej,
- 90% obsługi błędów,
- 85% integracji API.

System testów jest regularnie utrzymywany i rozszerzany wraz z rozwojem aplikacji, zapewniając wysoką jakość i niezawodność systemu. Regularne wykonywanie testów w procesie ciągłej integracji pozwala na szybkie wykrywanie i naprawę potencjalnych problemów, zanim dotrą one do użytkowników końcowych.

7.5 Porównanie wygenerowanych i oryginalnych partytur

Poniższe sekcje przedstawiają porównanie wygenerowanych partytur z oryginalnymi pod względem wysokości dźwięków. Wygenerowane partytury zostały stworzone przez model AI, podczas gdy oryginalne partytury to opublikowane wersje utworów. Znaczenie kolorów użytych w wygenerowanych partyturach:

- **Czerwony:** nieprawidłowy dźwięk (pod względem wysokości).
- **Zielony:** prawidłowa melodia (pod względem wysokości).
- **Niebieski:** prawidłowy akompaniament (pod względem wysokości).

7.5.1 Dokładność i ograniczenia transkrypcji

Analiza wygenerowanych transkrypcji wykazuje kilka charakterystycznych cech i ograniczeń systemu:

- **Wysokość dźwięków:** System wykazuje bardzo wysoką dokładność w identyfikacji wysokości dźwięków, co potwierdza skuteczność modelu w rozpoznawaniu częstotliwości.
- **Notacja rytmiczna:** Transkrypcje cechują się nadmierną dokładnością w zapisie rytmicznym. System skrupulatnie zapisuje wszelkie, nawet najmniejsze odchylenia tempa, nierówności rytmiczne oraz naturalne dla wykonania muzycznego *acceleranda* i *ritenuta*. Ta przesadna precyzja prowadzi do zaciemnienia obrazu utworu i utrudnia jego czytelność.
- **Pedalizacja:** System dosłownie interpretuje brzmienie dźwięków przedłużonych pedalem, zapisując je jako nuty przetrzymane przez cały czas ich realnego brzmienia. Prowadzi to do znacznego zagęszczenia notacji, podczas gdy w standardowym zapisie nutowym efekt ten osiąga się poprzez oznaczenie pedalizacji.
- **Organizacja metryczna:** System wykazuje pewne ograniczenia w zakresie właściwego doboru metrum oraz podziału na takty, co może prowadzić do nietypowych lub nieprawidłowych rozwiązań w zapisie metrycznym.
- **Dobór kluczy:** Zauważalne są również niedoskonałości w wyborze odpowiednich kluczy, co czasami prowadzi do mniej optymalnego zapisu nutowego niż w wersjach oryginalnych.

Powyższe cechy wskazują, że system, mimo wysokiej skuteczności w rozpoznawaniu wysokości dźwięków, wymaga dalszego rozwoju w zakresie tworzenia czytelnej i praktycznej notacji muzycznej, która lepiej odzwierciedlałaby konwencje przyjęte w tradycyjnym zapisie nutowym.

7.5.2 F. Chopin - Etiuda op. 10 nr 5

Wygenerowana partytura

[Link do wygenerowanej partytury](#)

Oryginalna partytura

[Link do oryginalnej partytury](#)

7.5.3 L. van Beethoven - Sonata op. 53 nr 21 “Waldstein”, Allegro con brio

Wygenerowana partytura

[Link do wygenerowanej partytury](#)

Oryginalna partytura

[Link do oryginalnej partytury](#)

7.5.4 L. van Beethoven - Sonata op. 13 nr 8 “Patetyczna”, Adagio cantabile

Wygenerowana partytura

[Link do wygenerowanej partytury](#)

Oryginalna partytura

[Link do oryginalnej partytury](#)

Handwritten musical score for piano, featuring six systems of staves (treble and bass clef). The score includes various musical notations such as notes, rests, and fingerings. The key signature is B-flat major (two flats). The tempo is marked $\text{♩} = 176$.

The score is divided into systems, with measures numbered 1 through 11. The notation includes various musical symbols, including notes, rests, and fingerings. The score is annotated with green and red highlights, likely indicating specific musical features or performance instructions.

System 1 (Measures 1-2): Measure 1 has a tempo marking $\text{♩} = 176$. Measure 2 has a fingering '5' above the treble staff.

System 2 (Measures 3-4): Measure 3 has a measure number '3' above the treble staff.

System 3 (Measures 5-6): Measure 5 has a measure number '5' above the treble staff.

System 4 (Measures 7-8): Measure 7 has a measure number '7' above the treble staff.

System 5 (Measures 9-10): Measure 9 has a measure number '9' above the treble staff.

System 6 (Measures 11-12): Measure 11 has a measure number '11' above the treble staff.

The score includes various musical notations, including notes, rests, and fingerings. The notation is written in a standard musical notation style, with treble and bass clefs. The key signature is B-flat major (two flats). The tempo is marked $\text{♩} = 176$.

The score is annotated with green and red highlights, likely indicating specific musical features or performance instructions. The green highlights are primarily in the treble staff, while the red highlights are primarily in the bass staff.

Vivace (♩ = 116)

brillante

Op. 10-Nr 5

5

1

f

(*Leg.* *)

p

Leg. *

legato

cresc.

Leg. *

4

f

p

(*Leg.* *)

Leg. *

(*Leg.* *)

Leg. *

8

poco rall.

a tempo

pp

f

p

cresc.

Leg. *

(*Leg.* *)

Leg. *

Leg. *

12

f

p

cresc.

Leg. *

Leg. *

Leg. *

Leg. *

16

p

cresc.

Leg. *

Leg. *

Leg. *

Leg. *

♩ = 158

This page of musical notation is for a piano piece, featuring two systems of grand staves (treble and bass clef). The tempo is marked as ♩ = 158. The notation includes various musical symbols such as notes, rests, and accidentals. Some notes are highlighted in green, and others in blue. The page is numbered 5, 8, 11, 14, and 16. The first system (measures 1-4) shows a complex rhythmic pattern in the bass clef with many beamed notes. The second system (measures 5-8) continues this pattern. The third system (measures 9-12) introduces a new melodic line in the treble clef. The fourth system (measures 13-16) features a more complex rhythmic pattern in the bass clef. The fifth system (measures 17-20) shows a continuation of the melodic line in the treble clef. The notation is written in a style that suggests a modern or contemporary composition.

SONATE

für das Pianoforte

Beethovens Werke.

von
L. VAN BEETHOVEN.

Serie 16. N^o 144.

Dem Grafen von Waldstein gewidmet.

Op. 53.

Allegro con brio.

Sonate N^o 21.

The musical score for Sonata N° 21 by Beethoven, Op. 53, 'Allegro con brio', is presented in six systems. Each system consists of a piano (piano) staff and a violin staff. The piano part is characterized by a continuous eighth-note accompaniment in the left hand and a more melodic line in the right hand. The violin part enters in the second system. Dynamics include *pp*, *cresc.*, *f*, *decresc.*, *p*, and *all.* The tempo is marked 'Allegro con brio'.

♩ = 117

First system of a musical score in 3/4 time, key of B-flat major. The left hand (bass clef) plays a steady eighth-note accompaniment, while the right hand (bass clef) plays a more complex melody. Green highlights are present in the right hand across measures 1-5, and blue highlights are in the left hand across measures 1-5. A red highlight is on the final note of measure 5 in the right hand.

Second system of the musical score (measures 6-10). The right hand continues its melody with green highlights, and the left hand provides accompaniment with blue highlights. A red highlight is on the final note of measure 10 in the right hand.

Third system of the musical score (measures 11-16). The right hand features a melodic line with green highlights, and the left hand has a more active accompaniment with blue highlights. A red highlight is on the final note of measure 16 in the right hand.

Fourth system of the musical score (measures 17-22). The right hand has a melodic line with green highlights, and the left hand has a more active accompaniment with blue highlights. A red highlight is on the final note of measure 22 in the right hand.

Fifth system of the musical score (measures 23-26). The right hand has a melodic line with green highlights, and the left hand has a more active accompaniment with blue highlights. A red highlight is on the final note of measure 26 in the right hand.

Sixth system of the musical score (measures 27-32). The right hand has a melodic line with green highlights, and the left hand has a more active accompaniment with blue highlights. A red highlight is on the final note of measure 32 in the right hand.

Adagio cantabile.

A musical score for a piano piece, marked "Adagio cantabile." The score is written for piano (p) and features a complex, flowing melody in the right hand and a more rhythmic, accompanimental line in the left hand. The key signature is three flats (B-flat, E-flat, A-flat), and the time signature is 4/4. The score is divided into six systems, each with two staves. The first system begins with a piano (p) dynamic marking. The melody in the right hand is characterized by long, sweeping lines and frequent grace notes. The left hand provides a steady accompaniment with eighth and sixteenth notes. The score concludes with a "cresc." (crescendo) marking and a final chord in the right hand.

p

cresc.

Rozdział 8

Podsumowanie i wnioski

W niniejszej pracy przedstawiono projekt i implementację aplikacji webowej pod nazwą TuneScript, umożliwiającej automatyczną transkrypcję nagrań fortepianowych z formatu audio (MP3, WAV) do notacji muzycznej w formacie PDF. W celu realizacji tego zadania wykorzystano zaawansowane modele sztucznej inteligencji, oparte na uczeniu głębokim.

Głównym założeniem projektu była budowa kompleksowego systemu, który łączy w sobie przetwarzanie sygnału dźwiękowego, generowanie plików MIDI, a następnie ich konwersję do czytelnej partytury. W realizacji tego celu uwzględniono następujące aspekty:

- **Architektura systemu:** Zastosowano podejście mikroservisowe, gdzie poszczególne komponenty (frontend, backend, worker, chatbot) działają niezależnie, komunikując się między sobą za pomocą protokołu HTTP i kolejki zadań asynchronicznych. Rozwiązanie takie zwiększa skalowalność i ułatwia wdrażanie zmian w systemie.
- **Model transkrypcji AI:** Istotą systemu jest model sieci neuronowej przetwarzającej mel-spektrogram nagrania i przewidującej początek i koniec dźwięków, ich wysokość, a także dynamikę. Rozwiązanie opiera się na wynikach prac Kong et al. (2020) [15], a także na wcześniejszych podejściach Hawthorne’a [10].
- **Interfejs użytkownika:** Zaimplementowano przyjazną dla użytkownika aplikację frontendową opartą na Next.js i TypeScript. Pozwala ona na przesyłanie plików audio, śledzenie procesu transkrypcji w czasie rzeczywistym, a także przeglądanie i ocenianie gotowych transkrypcji.
- **Weryfikacja i testy:** Przeprowadzono testy jednostkowe, integracyjne oraz manualne (m.in. import z YouTube, testy wydajności dla długich plików). System w obecnej postaci osiąga satysfakcjonujące wyniki, zarówno pod względem poprawności rozpoznawania wysokości dźwięków, jak i stabilności działania.

8.1 Ocena osiągniętych celów

Przedstawiony w rozdziale pierwszym cel główny — stworzenie kompletnego narzędzia do transkrypcji muzyki fortepianowej — został osiągnięty. Udało się zintegrować kluczowe funkcjonalności: przysyłanie plików audio, asynchroniczne przetwarzanie w workerze, generowanie i wizualizację wyników w postaci plików MIDI i PDF. Wysoka dokładność transkrypcji została potwierdzona w rozdziale dotyczącym walidacji, gdzie porównano wygenerowane partytury z oryginalnymi zapisami nutowymi.

Spośród celów szczegółowych udało się zrealizować wszystkie najistotniejsze:

- Opracowano model uczenia głębokiego do dokładnego rozpoznawania nut fortepianowych.
- Zbudowano wydajny serwis backendowy wraz z bazą danych (PostgreSQL) i systemem kolejkowania (Celery + Redis).
- Zapewniono narzędzia do generowania partytur PDF na podstawie plików MIDI.
- Stworzono intuicyjny interfejs webowy, ułatwiający pracę z transkrypcjami, w tym mechanizmy oceniania, wyszukiwania i dodawania do ulubionych.

8.2 Ograniczenia i problemy napotkane w trakcie pracy

W trakcie realizacji projektu zidentyfikowano kilka trudności i ograniczeń:

- **Zbyt szczegółowy zapis rytmiczny:** Model w niektórych przypadkach generuje nadmiernie dokładny zapis wszelkich niuansów wykonawczych (rubato, niewielkie przyspieszenia), co prowadzi do powstawania bardzo skomplikowanej notacji.
- **Pedalizacja fortepianowa:** Z uwagi na cechy nagrań fortepianowych, model dosłownie interpretuje przedłużenie dźwięków, co może utrudniać czytelność zapisu nutowego — w standardowej notacji stosuje się bowiem oznaczenia pedalizacji zamiast wielokrotnie przeciąganych nut.
- **Składnia nutowa (metrum i klucze):** Automatyczne dobieranie metrum i kluczy w zapisie wciąż wymaga dalszego udoskonalenia, zwłaszcza w utworach o nieregularnej strukturze rytmicznej lub częstej modulacji.
- **Problemy z kompatybilnością bibliotek:** W trakcie implementacji napotkano problem niekompatybilności wersji biblioteki PyTorch [27] między modulem chatbota a modelem transkrypcji. Problem ten wymusił modyfikację architektury systemu - chatbot został wydzielony jako osobny serwis i uruchomiony w oddzielnym kontenerze, co pozwoliło na niezależne zarządzanie zależnościami obu komponentów.

8.3 Możliwe kierunki dalszego rozwoju

Rozwijając dalej system TuneScript, można rozważyć następujące usprawnienia:

1. **Lepsza interpretacja rytmiczna:** Wprowadzenie dodatkowych algorytmów do „wygładzania” zapisu rytmicznego i uwzględniania konwencji notacji klasycznej.
2. **Zaawansowane oznaczenia wykonawcze:** Implementacja mechanizmów rozpoznawania artykulacji (*legato*, *staccato*), dynamicznych oznaczeń (*p*, *f*) i bardziej przyjazne przedstawianie użycia pedałów.
3. **Rozszerzenie na inne instrumenty:** Choć obecna architektura skupia się na fortepianie, możliwa jest adaptacja modelu do innych instrumentów (np. gitara czy skrzypce) lub do pełnej partytury orkiestrowej.
4. **Usprawnienia UX/UI:** Rozbudowa interfejsu o dodatkowe podpowiedzi, wizualizacje przebiegu transkrypcji w czasie czy personalizowane rekomendacje utworów dla użytkowników.
5. **Zastosowanie w edukacji:** Stworzenie trybu ćwiczebnego, w którym uczniowie mieliby dostęp do transkrypcji własnych nagrań i automatycznych wskazówek dotyczących techniki gry lub potencjalnych błędów w interpretacji.

8.4 Zakończenie

Podsumowując, praca osiągnęła wyznaczone cele. System TuneScript stanowi dowód na to, że najnowsze osiągnięcia w dziedzinie głębokiego uczenia maszynowego mogą być skutecznie wykorzystane w automatycznej transkrypcji muzyki fortepianowej, zapewniając użytkownikom wygodne narzędzie do analizy i zapisu nagrań w formacie PDF.

Mimo zauważonych ograniczeń, wyniki są na tyle obiecujące, że aplikacja ma realną szansę stać się wartościowym rozwiązaniem nie tylko dla muzyków i pedagogów, ale także dla szeroko pojętego środowiska zajmującego się przetwarzaniem informacji muzycznej. Dalsze prace pozwolą na rozszerzenie funkcjonalności, zwiększenie wygody użytkowników oraz poprawę jakości generowanego zapisu nutowego, tak by TuneScript mógł jeszcze skuteczniej służyć w codziennej praktyce muzycznej i edukacji.

Bibliografia

- [1] Apollo GraphQL. *Apollo Client Documentation*. 2024. URL: <https://www.apollographql.com/docs/react/> (term. wiz. 05.01.2024).
- [2] Emmanouil Benetos, Simon Dixon, Dimitrios Giannoulis, Holger Kirchhoff i Anssi Klapuri. „Automatic music transcription: challenges and future directions”. W: *Journal of Intelligent Information Systems* 41.3 (2013), s. 407–434.
- [3] Celery Project. *Celery Documentation*. 2024. URL: <https://docs.celeryq.dev/> (term. wiz. 05.01.2024).
- [4] Tian Cheng, Simon Dixon i Matthias Mauch. „Attack/decay analysis of piano sounds using an exponential model”. W: *Journal of the Acoustical Society of America* 139.6 (2016), s. 4332–4341.
- [5] Cypress. *Cypress Documentation*. 2024. URL: <https://docs.cypress.io/> (term. wiz. 05.01.2024).
- [6] Django Software Foundation. *Django Documentation*. 2024. URL: <https://docs.djangoproject.com/> (term. wiz. 05.01.2024).
- [7] Docker Inc. *Docker Documentation*. 2024. URL: <https://docs.docker.com/> (term. wiz. 05.01.2024).
- [8] ESLint. *ESLint Documentation*. 2024. URL: <https://eslint.org/docs/> (term. wiz. 05.01.2024).
- [9] Factory Boy. *Factory Boy Documentation*. 2024. URL: <https://factoryboy.readthedocs.io/> (term. wiz. 05.01.2024).
- [10] Curtis Hawthorne, Erich Elsen, Jialin Song, Adam Roberts, Ian Simon, Colin Raffel, Jesse Engel, Sageev Oore i Douglas Eck. „Onsets and frames: Dual-objective piano transcription”. W: *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*. Paris, France, wrz. 2018, s. 50–57.
- [11] Curtis Hawthorne, Andriy Stasyuk, Adam Roberts, Ian Simon, Cheng-Zhi Anna Huang, Sander Dieleman, Erich Elsen, Jesse Engel i Douglas Eck. „Enabling Factorized Piano Music Modeling and Generation with the MAESTRO Dataset”. W: *International Conference on Learning Representations (ICLR)*. New Orleans, USA, maj 2019. URL: <https://openreview.net/forum?id=r1lYRjC9F7>.

- [12] Hugging Face. *Transformers Documentation*. 2024. URL: <https://huggingface.co/docs/transformers/> (term. wiz. 05.01.2024).
- [13] Il-Young Jeong i Kyeongok Lee. „Deep Learning for Music”. W: *Proceedings of the 18th International Society for Music Information Retrieval Conference (ISMIR)*. Suzhou, China, paź. 2017, s. 463–469.
- [14] Rainer Kelz, Matthias Dorfer, Filip Korzeniowski, Sebastian Böck, Andreas Arzt i Gerhard Widmer. „On the Potential of Simple Framewise Approaches to Piano Transcription”. W: *Proceedings of the 17th International Society for Music Information Retrieval Conference (ISMIR)*. New York, USA, sierp. 2016, s. 475–481.
- [15] Qiuqiang Kong, Bochen Li, Xuchen Song, Yuan Wan i Yuxuan Wang. „High-resolution Piano Transcription with Pedals by Regressing Onsets and Offsets Times”. W: *arXiv preprint arXiv:2010.01815* (2020).
- [16] Sonali Mathur i Shaily Malik. „Advancements in the V-Model”. W: *International Journal of Computer Applications* 1.12 (2010), s. 29–34.
- [17] Meta Open Source. *Jest Documentation*. 2024. URL: <https://jestjs.io/docs/getting-started> (term. wiz. 05.01.2024).
- [18] Microsoft. *TypeScript Documentation*. 2024. URL: <https://www.typescriptlang.org/docs/> (term. wiz. 05.01.2024).
- [19] Pallets Projects. *Flask Documentation*. 2024. URL: <https://flask.palletsprojects.com/en/> (term. wiz. 05.01.2024).
- [20] PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2024. URL: <https://www.postgresql.org/docs/> (term. wiz. 05.01.2024).
- [21] Prettier. *Prettier Documentation*. 2024. URL: <https://prettier.io/docs/en/> (term. wiz. 05.01.2024).
- [22] PyCQA. *Flake8 Documentation*. 2024. URL: <https://flake8.pycqa.org/en/latest/> (term. wiz. 05.01.2024).
- [23] pytest team. *pytest Documentation*. 2024. URL: <https://docs.pytest.org/> (term. wiz. 05.01.2024).
- [24] Python Poetry. *Poetry Documentation*. 2024. URL: <https://python-poetry.org/docs/> (term. wiz. 05.01.2024).
- [25] Python Software Foundation. *Black Documentation*. 2024. URL: <https://black.readthedocs.io/> (term. wiz. 05.01.2024).
- [26] Python Software Foundation. *Python Documentation*. 2024. URL: <https://docs.python.org/3/> (term. wiz. 05.01.2024).

- [27] PyTorch Team. *PyTorch Documentation*. 2024. URL: <https://pytorch.org/docs/> (term. wiz. 05.01.2024).
- [28] Redis Ltd. *Redis Documentation*. 2024. URL: <https://redis.io/documentation> (term. wiz. 05.01.2024).
- [29] Siddharth Sigtia, Emmanouil Benetos i Simon Dixon. „An end-to-end neural network for polyphonic piano music transcription”. W: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 24.5 (2016), s. 927–939.
- [30] Testing Library. *React Testing Library Documentation*. 2024. URL: <https://testing-library.com/docs/react-testing-library/intro/> (term. wiz. 05.01.2024).
- [31] Timothy Crosley. *isort Documentation*. 2024. URL: <https://pycqa.github.io/isort/> (term. wiz. 05.01.2024).
- [32] Vercel Inc. *Next.js Documentation*. 2024. URL: <https://nextjs.org/docs> (term. wiz. 05.01.2024).

Dodatki

Źródła

Listing 1: Model regresji dla momentu naciśnięcia i puszczenia klawisza

```
1  class RegressOnsetOffsetFrameVelocityCRNN(nn.Module):
2      def __init__(self, frames_per_second, classes_num):
3          super().__init__()
4
5          window_size = 2048
6          hop_size = sample_rate // frames_per_second
7          mel_bins = 229
8
9          # Ekstraktor cech spektralnych
10         self.spectrogram_extractor = Spectrogram(
11             n_fft=window_size,
12             hop_length=hop_size,
13             win_length=window_size,
14             window='hann',
15             center=True,
16             pad_mode='reflect',
17             freeze_parameters=True)
18
19         # Ekstraktor cech mel-spektrogramu
20         self.logmel_extractor = LogmelFilterBank(
21             sr=sample_rate,
22             n_fft=window_size,
23             n_mels=mel_bins,
24             fmin=30,
25             fmax=sample_rate // 2,
26             freeze_parameters=True)
27
28         # Modele do detekcji różnych aspektów
29         self.frame_model = AcousticModelCRnn8Dropout(
30             classes_num, midfeat, momentum)
31         self.reg_onset_model = AcousticModelCRnn8Dropout(
32             classes_num, midfeat, momentum)
33         self.reg_offset_model = AcousticModelCRnn8Dropout(
34             classes_num, midfeat, momentum)
35         self.velocity_model = AcousticModelCRnn8Dropout(
36             classes_num, midfeat, momentum)
37
38         # GRU do przetwarzania sekwencyjnego
39         self.reg_onset_gru = nn.GRU(
40             input_size=88 * 2,
41             hidden_size=256,
42             num_layers=1,
43             batch_first=True,
44             bidirectional=True)
```

Listing 2: Asynchroniczne zadanie do transkrypcji

```

1  @shared_task(bind=True, max_retries=3)
2  def process_transcription(self, transcription_id):
3      try:
4          transcription = Transcription.objects.get(pk=
5              transcription_id)
6          audio_file = transcription.audio_file
7
8          # Wczytanie i preprocessing audio
9          audio, _ = load_audio(
10              audio_file.audio_file.path,
11              sr=sample_rate,
12              mono=True)
13
14          # Inicjalizacja modelu transkrypcji
15          transcriptor = PianoTranscription(device='cpu')
16
17          # Generowanie pliku MIDI
18          with tempfile.NamedTemporaryFile(
19              delete=False, suffix='.mid') as tmp_midi:
20              midi_path = tmp_midi.name
21              transcriptor.transcribe(audio, midi_path)
22
23          # Zapisywanie pliku MIDI
24          with open(midi_path, 'rb') as midi_file:
25              midi_instance = MIDIFile(transcription=transcription)
26              midi_instance.midi_file.save(
27                  f"{transcription.title}.mid",
28                  File(midi_file))
29
30          # Generowanie PDF z MIDI
31          pdf_path = os.path.splitext(midi_path)[0] + '.pdf'
32          convert_midi_to_pdf(midi_data, pdf_path)
33
34          # Zapisywanie pliku PDF
35          with open(pdf_path, 'rb') as pdf_file:
36              sheet_music = SheetMusic(transcription=transcription)
37              sheet_music.pdf_file.save(
38                  f"{transcription.title}.pdf",
39                  File(pdf_file))
40
41          transcription.status = 'COMPLETED'
42          transcription.save()
43
44      except Exception as e:
45          transcription.status = 'FAILED'
46          transcription.save()
47          raise self.retry(exc=e, countdown=60)

```

Listing 3: Zdarzenia wysyłane przez serwer

```

1  @require_GET
2  @csrf_exempt
3  def sse_stream(request, transcription_id):
4      def event_stream():
5          while True:
6              try:
7                  _, db_id = from_global_id(transcription_id)
8                  transcription = Transcription.objects.get(pk=db_id
9
10
11                 # Przygotowanie danych o statusie
12                 data = {
13                     'transcription_id': transcription_id,
14                     'status': transcription.status,
15                     'title': transcription.title,
16                     'midi_file_url': (
17                         f"{settings.BASE_URL}{midi_file.midi_file.
18                             url}"
19                         if transcription.midifile_set.exists()
20                         else None
21                     ),
22                     'sheet_music_url': (
23                         f"{settings.BASE_URL}{sheet_music.pdf_file
24                             .url}"
25                         if transcription.sheetmusic_set.exists()
26                         else None
27                     )
28                 }
29
30                 yield f"data: {json.dumps(data)}\n\n"
31
32                 if transcription.status in ['COMPLETED', 'FAILED']:
33                     break
34
35                 time.sleep(2) # Sprawdzanie co 2 sekundy
36
37             except Transcription.DoesNotExist:
38                 yield f"data: {json.dumps({'error': 'Not found'})
39                     }\n\n"
40                 break
41
42         response = StreamingHttpResponse(
43             event_stream(),
44             content_type='text/event-stream')
45         response['Cache-Control'] = 'no-cache'
46         response['X-Accel-Buffering'] = 'no'
47         return response

```

Spis rysunków

2.1	Architektura systemu TuneScript	9
3.1	Diagram przypadków użycia systemu TuneScript	17
4.1	Strona główna systemu TuneScript	22
4.2	Panel główny użytkownika	22
4.3	Ekran przesyłania pliku audio	23
4.4	Status procesu transkrypcji	23
4.5	Zakończenie procesu transkrypcji	24
4.7	Lista transkrypcji użytkownika	24
4.6	Szczegółowy widok transkrypcji	25
4.8	Wyszukiwanie transkrypcji	26
4.10	Ustawienia preferencji użytkownika	26
4.9	Profil użytkownika	27
4.11	Lista ulubionych transkrypcji	28
4.12	Asystent chatbot	28
5.1	Schemat bazy danych TuneScript	32
5.2	Diagram klas modułu RAG	35
5.3	Diagram klas modułu transkrypcji muzycznej	35
6.1	Diagram architektury modelu transkrypcji	38

Spis tabel

7.1	Wyniki testów przetwarzania plików	46
-----	--	----